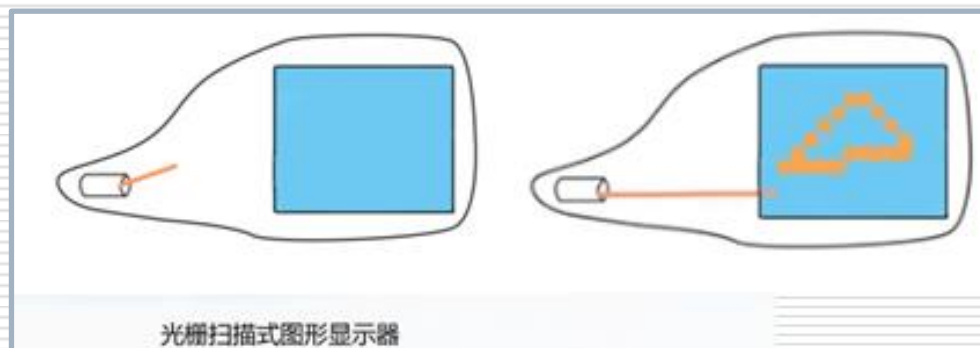

Rasterizing (光栅化) Graphics

Contents

- ❑ 扫描转换(**Scan Conversion**)的概念
 - ❑ 直线段(**Line**)的扫描转换
 - ❑ 圆(**Circle**)的扫描转换
 - ❑ 多边形(**Polygon**)的扫描转换与区域填充
 - ❑ 字符(**Character**)处理
 - ❑ 在**OpenGL**中绘制图形
-

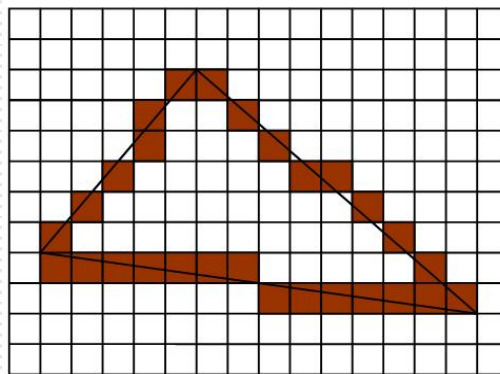
Scan Conversion

- **20世纪70年代**，光栅扫描式图形显示器广泛应用。这种显示器有一个**点阵单元**发生器，通过控制每个点阵单元的亮度来显示一幅完整的图形。
- 以此设备为基础，光栅图形学迅速发展，图元生成、区域填充、裁剪、反走样、消隐等概念和算法纷纷诞生。



Scan Conversion

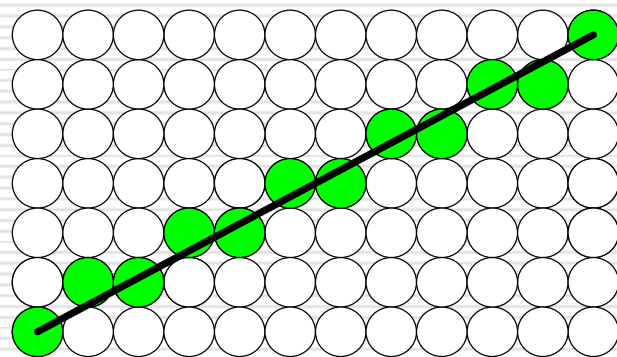
- 和**CRT**相似，**LCD**(液晶显示器)，也都有一个点阵单元发生器，该点阵单元其实就是 **像素矩阵**。
- 在这样的一个设备上生成图形的过程其实就是一个光栅化的过程。光栅化（也称为**扫描转换**）的过程可以分成两步：
第一步，根据图形的定义在点阵单元上确定**最佳逼近于图形**的**像素集**，逼近的过程本质是连续量向离散量的转换。
第二步，给像素指定合适的颜色值。



基本图形的扫描转换

(Generation of Basic primitives)

- 图形的生成：是在指定的输出设备上，根据坐标描述构造二维几何图形（点、直线、圆、椭圆、多边形等）。
- 由于光栅显示器的特点，显示屏上的每幅画面均由离散的像素构成，一条直线只能用一系列靠近直线的像素近似地表示。
- 图形的扫描转换：在光栅显示器等数字设备上确定一个最佳逼近于图形的像素集的过程。



图：用像素点集逼近直线

光栅图形学过时了吗？

- ✓ 光栅化并未过时，它仍是现代图形学的基石，与光线追踪共同构成当前及未来的主流渲染架构。

光栅图形学的核心是：将三维空间的几何图元(三角形、线段)通过一系列变换和离散化，映射到由像素构成的二维屏幕上，并决定每个像素的颜色。

- 光栅图形学：相当于地基、钢筋混凝土和砖瓦。它教你如何把几何体变成屏幕上一个一个有颜色的像素。
- 高级图形学：相当于装修、智能家居和摩天大楼设计。它建立在坚固的地基之上。

1. 光栅化：实时渲染的绝对主流

- 占据当前实时渲染的 99% 以上
- 应用领域：3A游戏、手机游戏、AR/VR、数字孪生、工业设计实时预览

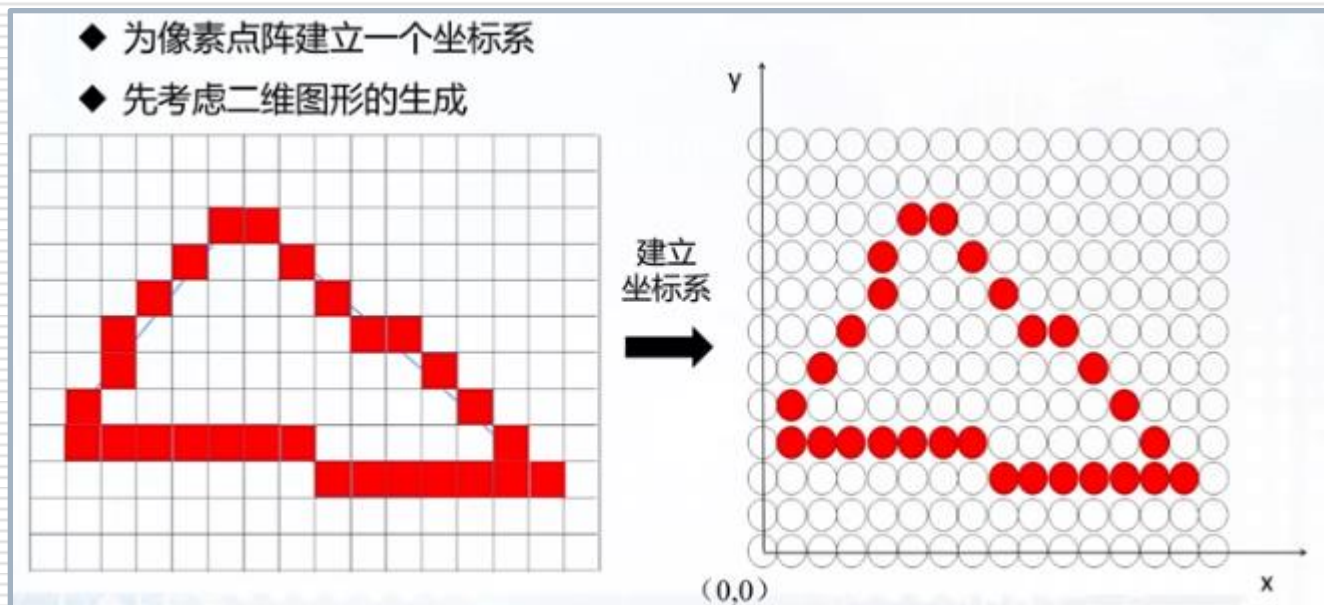
2. 光线追踪：增强，而非替代

- 传统光栅化：负责基础画面（速度快）
- 光线追踪：负责精确反射、折射、阴影（效果好，但速度慢）
- 当前主流方案：**混合渲染** → 兼顾画质与帧率

光栅化负责“基础画面”
光线追踪负责“锦上添花”

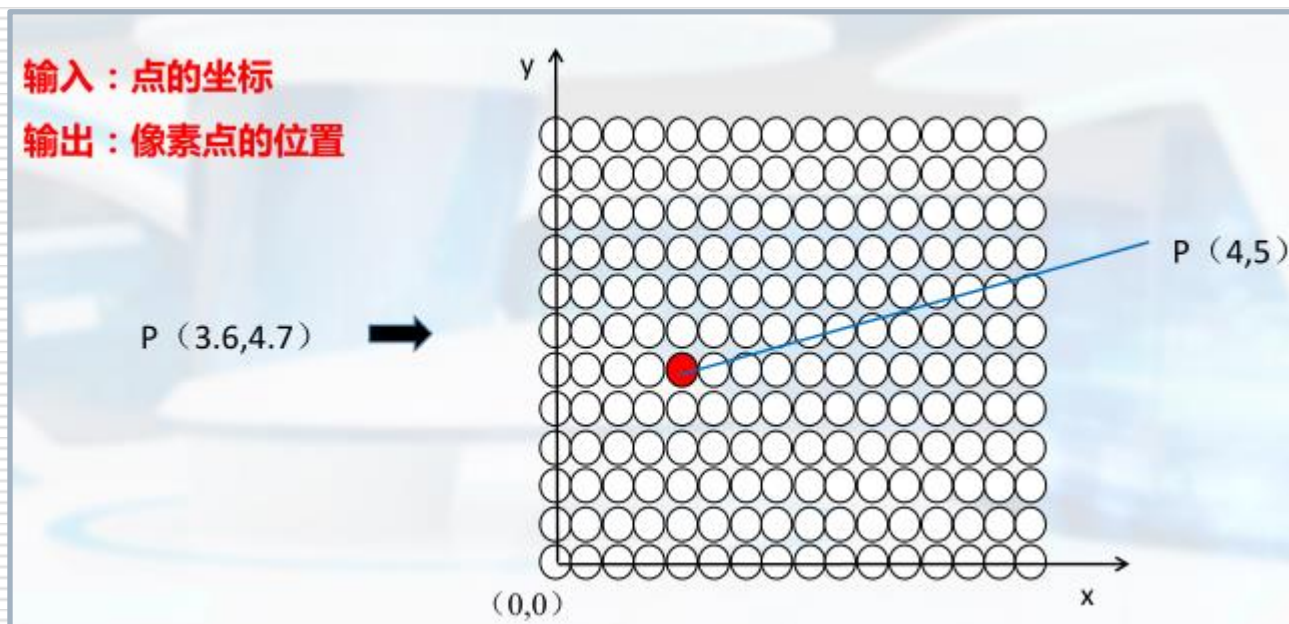
基本图形的扫描转换 (Generation of Basic primitives)

- 约定，为**像素点阵**建立一个坐标系，左下角为原点，很显然，每个像素点的**x**和**y**坐标都必须是整型的数值。



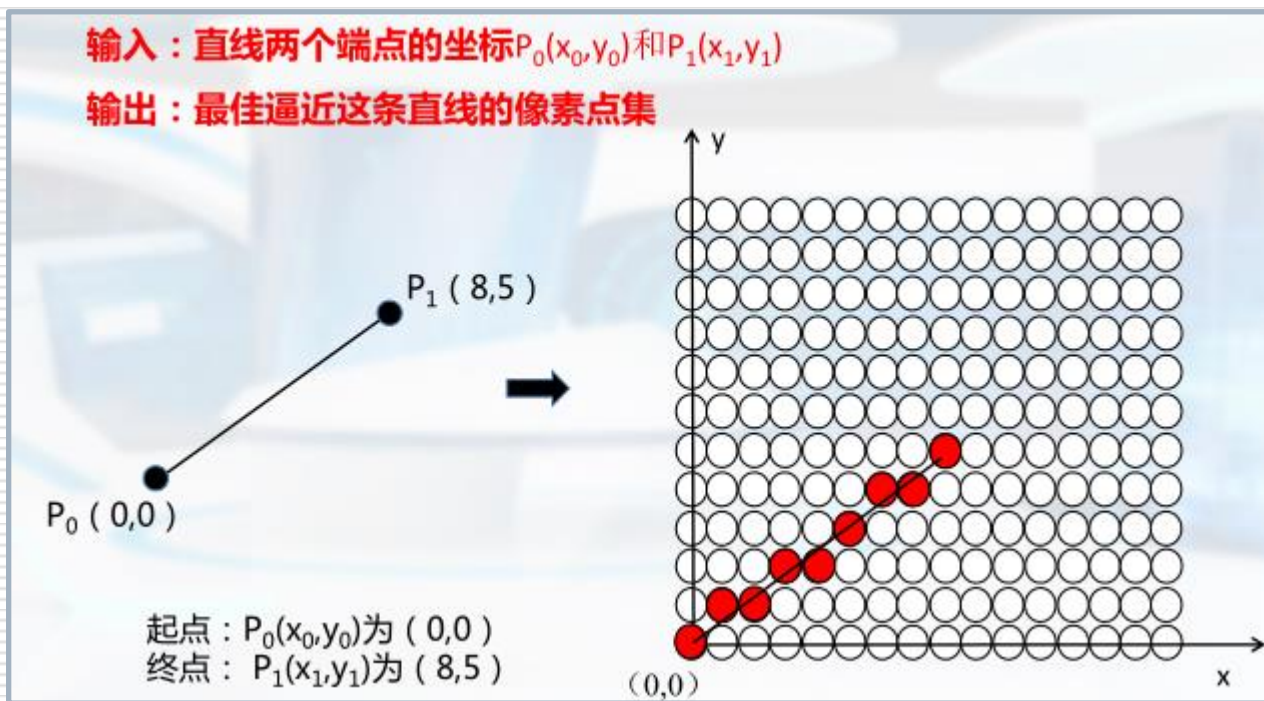
基本图形的扫描转换-Point

□ 点，输出最佳逼近这个点的像素点



基本图形的扫描转换-Line

□ 直线



直线的扫描转换 (Line's generation)

□ 高质量的直线的绘制要求

(1) 直线要直：像素点要尽量靠近理想的直线；反走样尽量减少锯齿现象。

(2) 直线的端点要准确，无定向性无断裂：从A端点到B端点绘制的直线与从B到A绘制的直线应该重合；另外由算法的累计误差和设备精度所致产生的断裂、不连接情况也是不允许的。

(3) 直线的亮度、色泽要均匀：若绘制出的像素点密度不均匀，就从视觉上给人一段亮，一段暗的感觉

(4) 画线的速度要快。

直线的扫描转换 (Line's generation)

- ✓ 解决的问题：给定直线两端点 $P_0(x_0, y_0)$ 和 $P_1(x_1, y_1)$ ，画出该直线。
- 数值微分法 (DDA, Digital Differential Analyzer)
- 中点 Bresenhan 算法 (中点画线算法)
- 改进的 Bresenhan 算法 (Bresenhan 算法)

数值微分法 (DDA, Digital Differential Analyzer)

□ **DDA**算法是根据直线的微分方程来生成直线的算法。

已知起点 $P_0(x_0, y_0)$ 和终点 $P_1(x_1, y_1)$,直线的微分方程如下:

$$\frac{dy}{dx} = \frac{\Delta y}{\Delta x} = \frac{y_1 - y_0}{x_1 - x_0} = k$$

□ **DDA**算法原理: 如右图:

(由于直线的一阶导数是连续的, 而且对于 Δx 与 Δy 是成比例的, 因此可以通过当前位置在 x 和 y 方向上分别加一个小增量来求出下一个点)

$$x_{i+1} = x_i + \varepsilon \cdot \Delta x$$

$$y_{i+1} = y_i + \varepsilon \cdot \Delta y$$

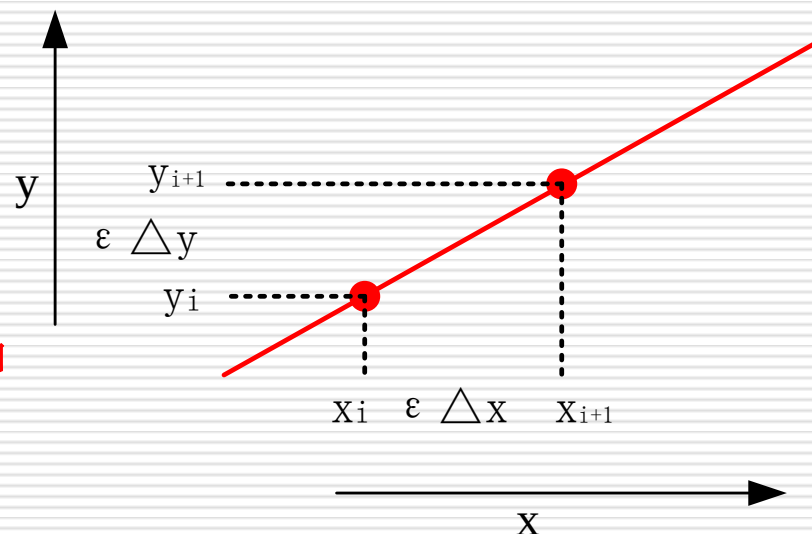
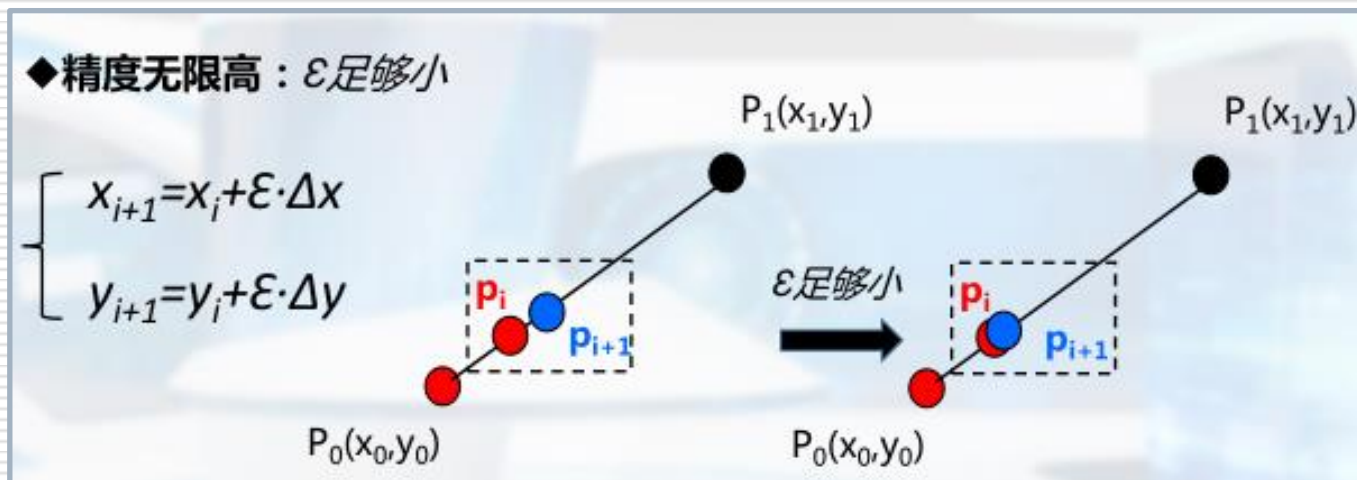


图: DDA算法原理

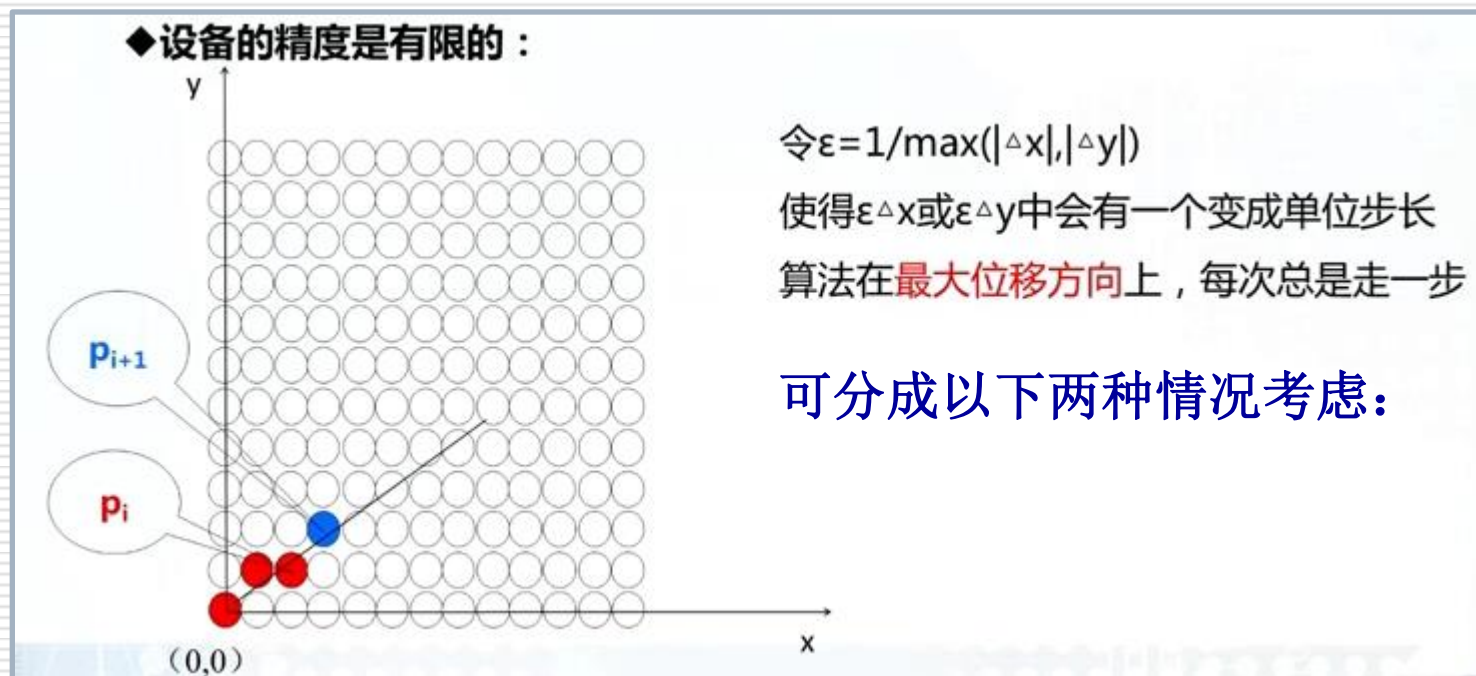
数值微分法(DDA法)

□ 该方法在精度无限高情况下可以绘制出无误差的直线



数值微分法(DDA法)

□ 但是设备精度有限



□ 最大位移方向是 $\mathbf{x}$ 方向, $\max(|\Delta x|, |\Delta y|) = |\Delta x|$,
即 $|k| \leq 1$ 的情况:

$$x_{i+1} = x_i + \varepsilon \cdot \Delta x = x_i + \frac{1}{|\Delta x|} \cdot \Delta x = x_i \pm 1$$

$$y_{i+1} = y_i + \varepsilon \cdot \Delta y = y_i + \frac{1}{|\Delta x|} \cdot \Delta y = y_i \pm k$$

y的增量是k

□ 最大位移方向是 $\mathbf{y}$ 方向, $\max(|\Delta x|, |\Delta y|) = |\Delta y|$,
此时 $|k| \geq 1$:

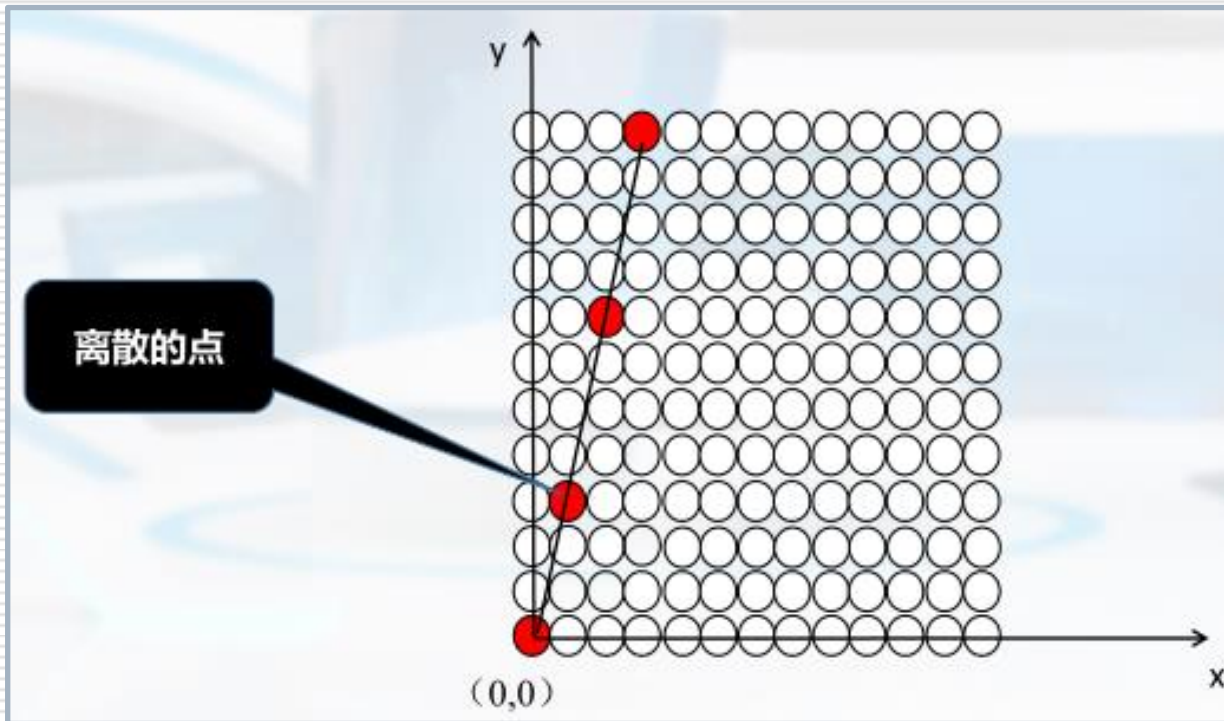
x的增量是1/k

$$x_{i+1} = x_i + \varepsilon \cdot \Delta x = x_i + \frac{1}{|\Delta y|} \cdot \Delta x = x_i \pm \frac{1}{k}$$

$$y_{i+1} = y_i + \varepsilon \cdot \Delta y = y_i + \frac{1}{|\Delta y|} \cdot \Delta y = y_i \pm 1$$

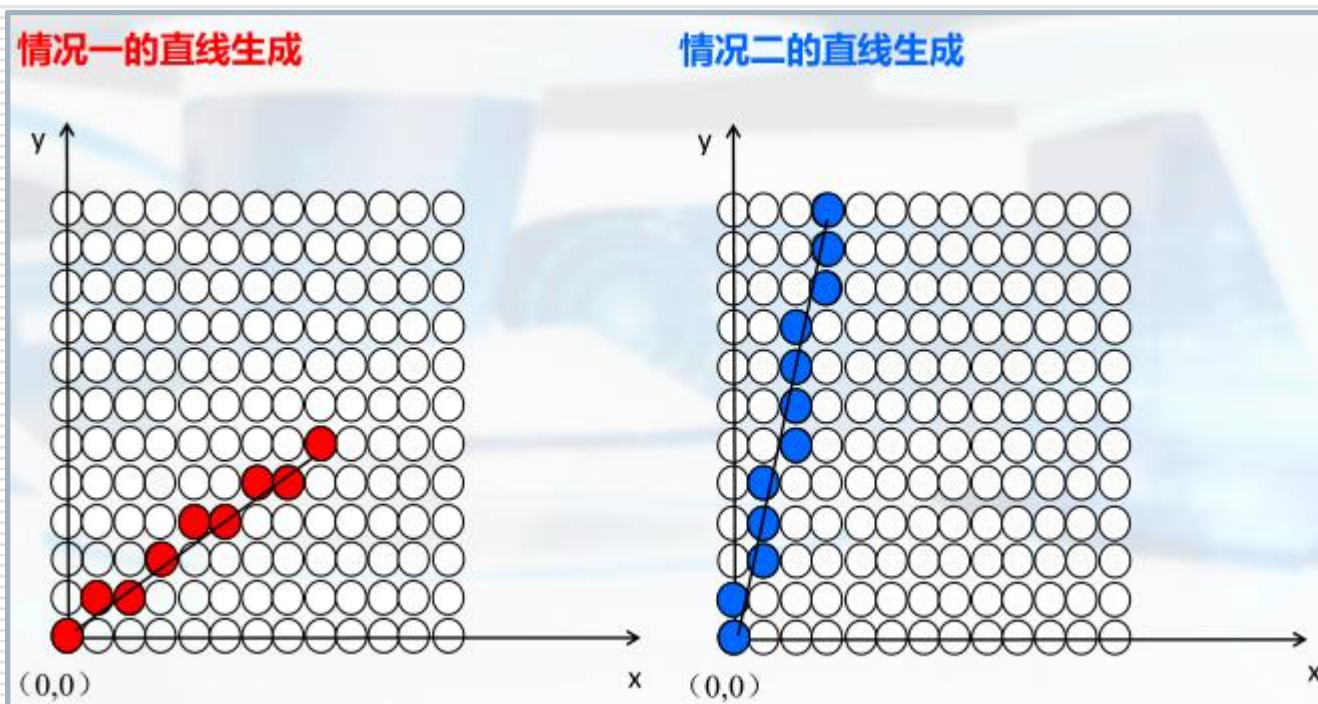
数值微分法(DDA法)

□ 最大位移方向的重要性



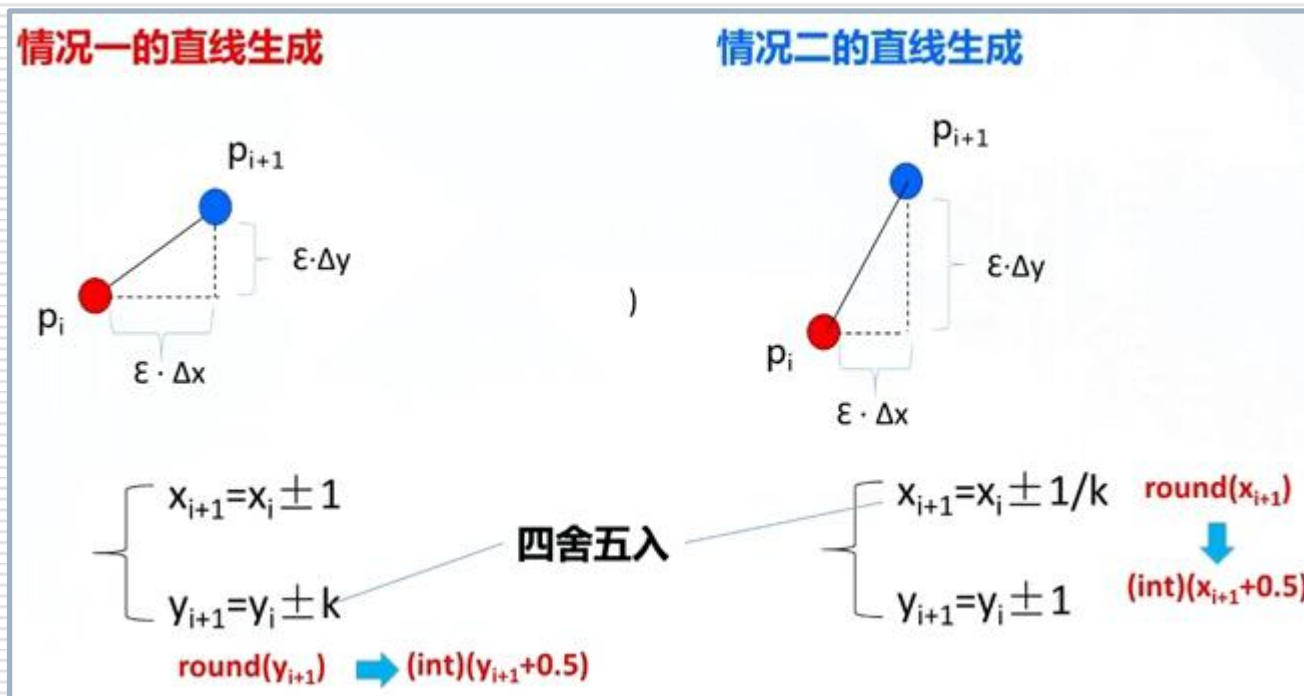
数值微分法(DDA法)

□ 两种情况下直线的生成



数值微分法(DDA法)

- 由于光栅化过程中不可能绘制半个像素点，需要对求得的值进行四舍五入。



数值微分法(DDA法)

思考：为什么不直接按照 $y=kx+b$ 计算相应的 y 坐标呢？

```
void DDALine(int x0,int y0,int x1,int y1)
{
    int  dx,dy,eps1,k;
    float x,y,xIncre,yIncre;
    dx=x1-x0; dy=y1-y0;
    x=x0;    y=y0;
    If (abs(dx)>abs(dy)) eps1=abs(dx);
    else eps1=abs(dy);
    xIncre=(float)dx/(float)eps1;
    yIncre=(float)dy/(float)eps1;
    for (k=0;k<=eps1;k++)
    {
        putpixel((int)(x+0.5),(int)(y+0.5));
        x+=xIncre;
        y+=yIncre;
    }
}
```

输出最佳逼近的像素点

最大位移方向的判断

x、y方向上增量的计算

OpenGL中用`glVertex2f`画点。

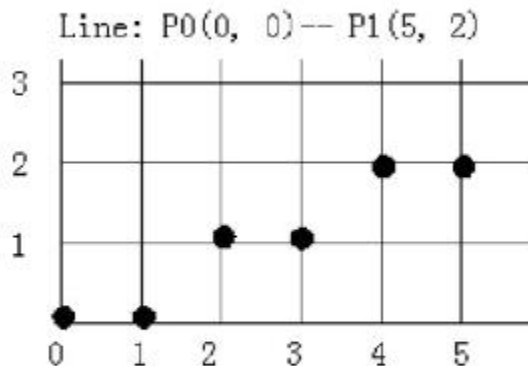
Turbo C 的图形函数，头文件`graphics.h`包含`putpixel()`函数，该函数在指定颜色的位置(x, y)处绘制像素。

Instance

教材P20例2.1:

用DDA算法扫描转换两点P0(0,0)到P1(5,2)的直线段。

x	int(y+0.5)	y+0.5
0	0	0
1	0	0.4+0.5
2	1	0.8+0.5
3	1	1.2+0.5
4	2	1.6+0.5



当 $x=1$; 有 $y_{i+1} = y_i + k$. 这里: $k = (2-0)/(5-0) = 0.4$

数值微分法(DDA法)——特点

- 增量算法：在一个迭代算法中，如果每一步的 x ， y 是用前一步的值加上一个增量来获得的，那么这种算法就称为增量算法，因此，**DDA**是一个增量算法。
- 直观、易实现
- 不利于用硬件实现
 - ☞ 需要取整操作和浮点计算（ x 和 y 方向增量的计算和浮点数相加），消耗时间。
 - ☞ 浮点数相加积累误差，对长线段而言，引起像素点位置与理想位置的偏移。

中点Bresenham算法(中点画线法)

MidPoint Bresenham Algorithm

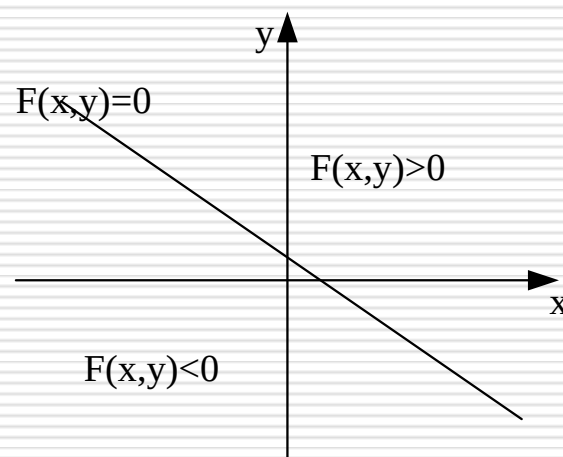
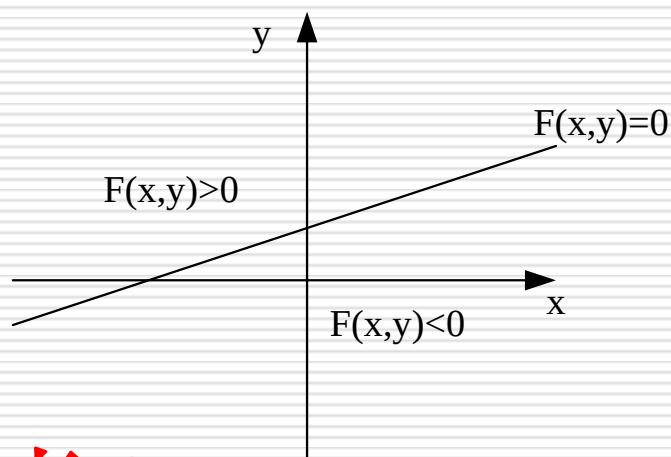
- 算法原理：根据直线的斜率确定或选择变量在x或y方向上每次递增一个单位，而另一方向的增量为1或0，它取决于实际直线与相邻像素点的距离，这一距离称为误差项。
- 然后根据误差项来确定另一个方向是否走步。这就是中点Bresenham算法。

中点Bresenham算法

MidPoint Bresenham Algorithm

□ 直线的隐式方程（直线的一般式方程）

$$F(x, y) = y - kx - b = 0, \text{ 其中 } k = \frac{\Delta y}{\Delta x} = \frac{y_1 - y_0}{x_1 - x_0}$$



思考？

图:直线将平面分为三个区域

□ 直线段的隐式方程 $((x_0, y_0), (x_1, y_1))$ 两端点

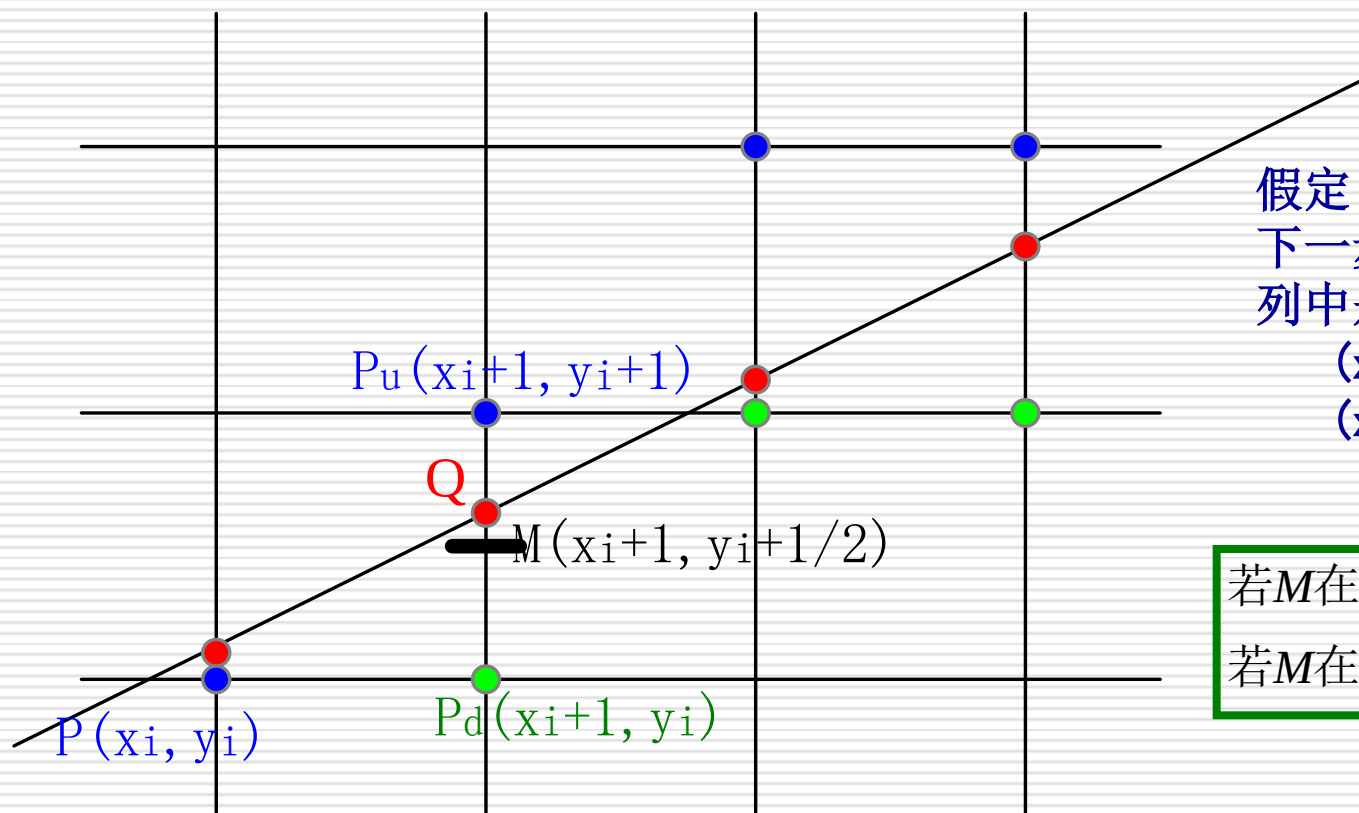
$$F(x, y) = ax + by + c = 0$$

式中 $a = y_0 - y_1, b = x_1 - x_0, c = x_0 y_1 - x_1 y_0$

也可以, 区别是乘 Δx

假定 $0 \leq k \leq 1$,

即 $0 \leq \Delta y / \Delta x \leq 1$, x 是最大位移方向



假定 (x_i, y_i) 已经被显示。
下一步要确定的是在 x_{i+1}
列中是哪个候选点被显示
 (x_{i+1}, y_i) ?
 (x_{i+1}, y_{i+1}) ?

若 M 在 Q 下方,即 $y_M < y_Q$,选 P_u
若 M 在 Q 上方, $y_M > y_Q$,选 P_d

图: Bresenham算法生成直线的原理

如何判断 Q 在 M 的上方还是下方? 一只需要判断 M 是在直线的上方还是下方, 于是隐式方程就用上了!

判别式：定义误差项 d

$$d = F(x_M, y_M) = F(x_i + 1, y_i + 0.5) = y_i + 0.5 - k(x_i + 1) - b$$

复杂的
计算过程

可以对误差项进行递推来简化算法

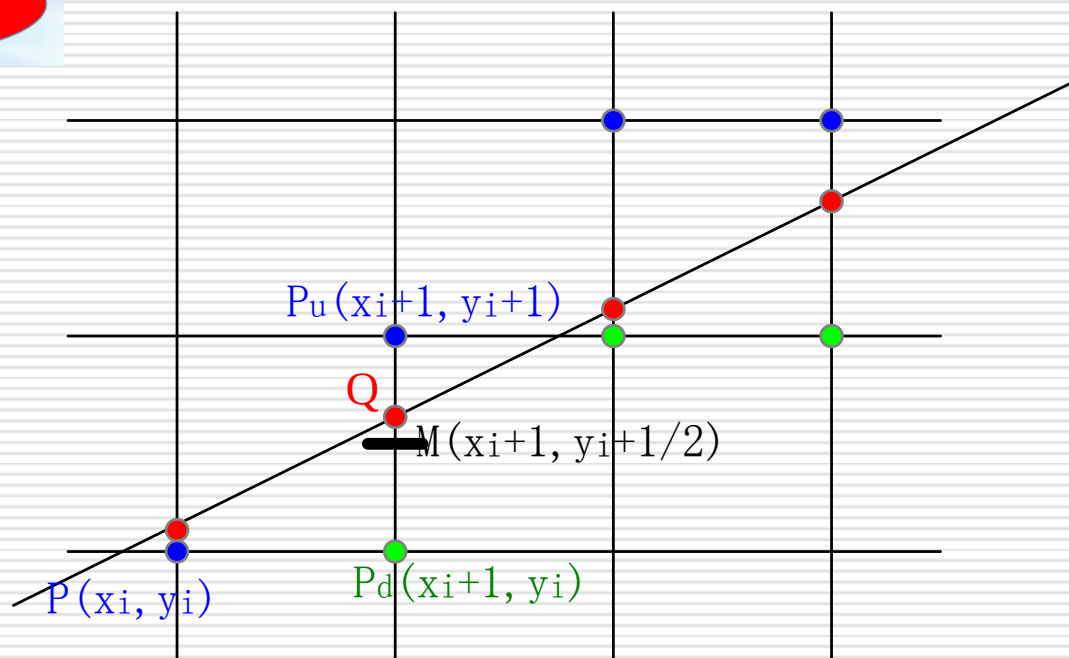
则有：

$$\begin{cases} x_{i+1} = x_i + 1 \\ y_{i+1} = \begin{cases} y_i + 1 & (d < 0) \\ y_i & (d \geq 0) \end{cases} \end{cases}$$

$$d > 0 \quad y_M > y_Q$$

$$d < 0 \quad y_M < y_Q$$

$$d = 0 \quad y_M = y_Q$$



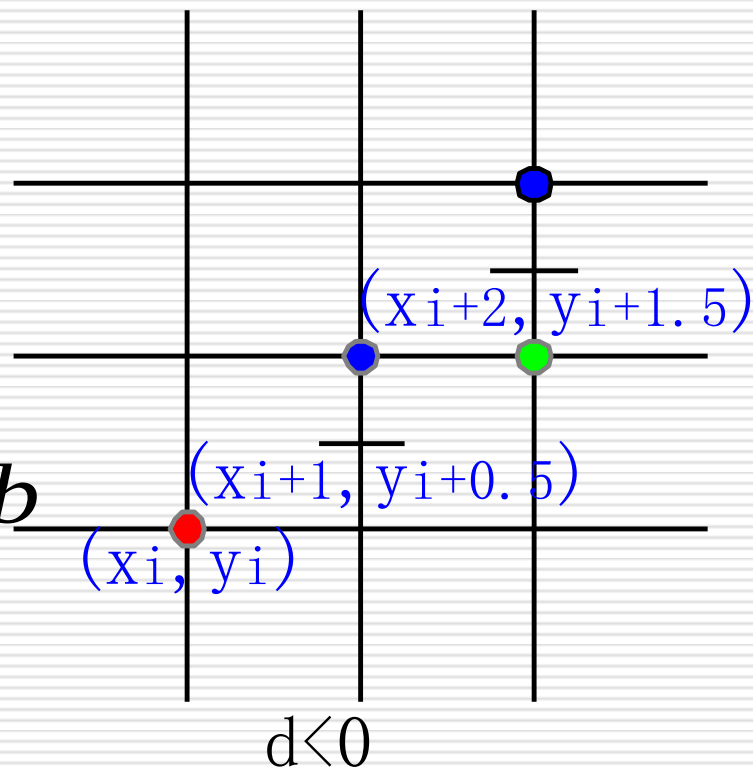
图：Brensemham算法生成直线的原理

误差项的递推 ($d < 0$)

$$\begin{aligned}d_1 &= F(x_i + 1, y_i + 0.5) \\ &= y_i + 0.5 - k(x_i + 1) - b\end{aligned}$$

$$\begin{aligned}d_2 &= F(x_i + 2, y_i + 1.5) \\ &= y_i + 1.5 - k(x_i + 2) - b\end{aligned}$$

$$\begin{aligned}d_2 &= y_i + 0.5 - k(x_i + 1) - b + 1 - k \\ &= d_1 + 1 - k\end{aligned}$$



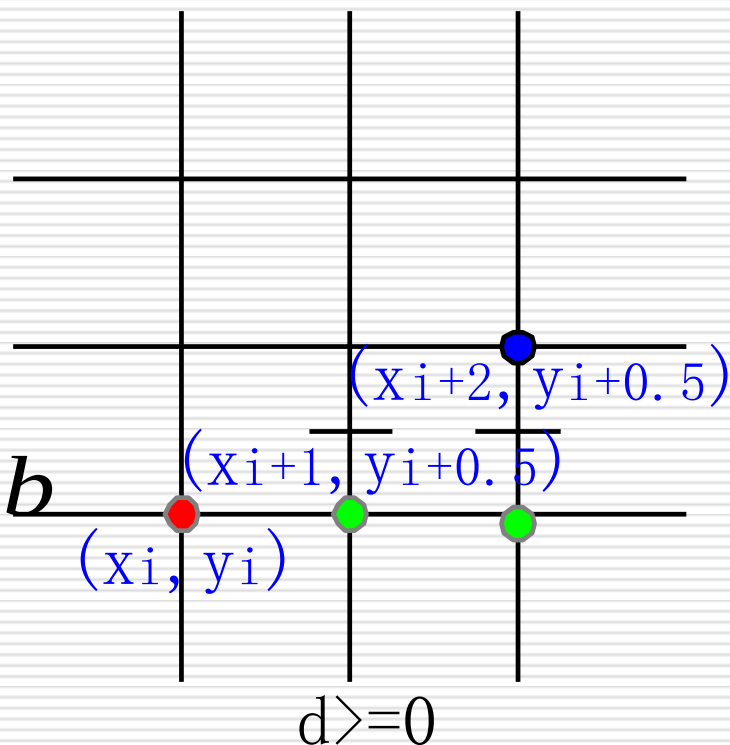
图：误差项递推

误差项的递推 ($d \geq 0$)

$$\begin{aligned}d_1 &= F(x_i + 1, y_i + 0.5) \\&= y_i + 0.5 - k(x_i + 1) - b\end{aligned}$$

$$\begin{aligned}d_2 &= F(x_i + 2, y_i + 0.5) \\&= y_i + 0.5 - k(x_i + 2) - b\end{aligned}$$

$$\begin{aligned}d_2 &= y_i + 0.5 - k(x_i + 1) - b - k \\&= d_1 - k\end{aligned}$$



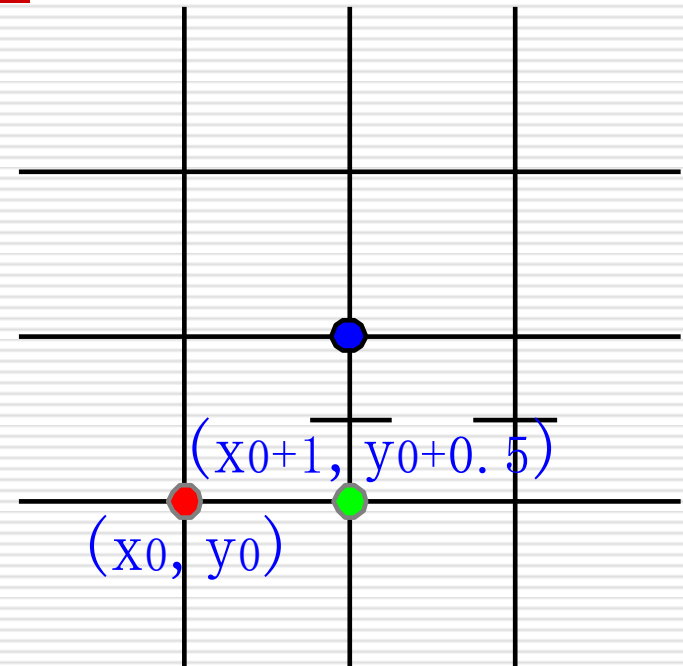
图：误差项递推

中点Bresenham算法

MidPoint Bresenham Algorithm

初始值d的计算

$$\begin{aligned}d_0 &= F(x_0 + 1, y_0 + 0.5) \\&= y_0 + 0.5 - k(x_0 + 1) - b \\&= y_0 - kx_0 - b - k + 0.5 \\&= 0.5 - k\end{aligned}$$



图： 计算初值

中点Bresenham算法

MidPoint Bresenham Algorithm

中点Bresenham算法

比DDA好吗？

d 的计算变成了增量计算
可是.....

d 的初值

$$\begin{aligned}d_0 &= F(x_0 + 1, y_0 + 0.5) \\&= y_0 + 0.5 - k(x_0 + 1) - b \\&= y_0 - kx_0 - b - k + 0.5 \\&= 0.5 - k\end{aligned}$$

浮点数

$$\begin{aligned}d < 0 \quad d &= F(x_i + 2, y_i + 1.5) \\&= y_i + 1.5 - k(x_i + 2) - b \\&= y_i + 1.5 - k(x_i + 1) - b - k \\&= d + 1 - k\end{aligned}$$

$$\begin{aligned}d \geq 0 \quad d &= F(x_i + 2, y_i + 0.5) \\&= y_i + 0.5 - k(x_i + 2) - b \\&= y_i + 0.5 - k(x_i + 1) - b - k \\&= d - k\end{aligned}$$

- 仍然没有超越DDA：其中有浮点数及浮点数运算，如 k 和 d 。
- 如何去掉小数呢？
 - ✓ 我们只用到 d 的符号。
 - ✓ 把 d 放大正整数倍，把它变成整数，并不影响符号。
- 放大多少倍呢？

中点Bresenham算法

MidPoint Bresenham Algorithm

0.5 $\rightarrow \times 2$

$k = \Delta y / \Delta x$ $\rightarrow \times \Delta x$

$\rightarrow$ 将d放大 $2\Delta x$ 倍

MidPoint Bresenham Algorithm

——Improve

改进：用 $2d\Delta x$ 代替 d ，令 $D=2d\Delta x$ ， D 与 d 同号，

则： $D_0 = 2\Delta x d_0 = 2\Delta x(0.5 - k) = \Delta x - 2\Delta y$

$$d < 0 \quad D < 0$$

$$D = 2\Delta x d = 2\Delta x(d + 1 - k)$$

$$= 2\Delta x d + 2\Delta x - 2\Delta y$$

$$= D + 2\Delta x - 2\Delta y$$

$$d \geq 0 \quad D \geq 0$$

$$D = 2\Delta x d = 2\Delta x(d - k) = D - 2\Delta y$$

$$D = 2d\Delta x$$

改用整数
便于硬件计算

MidPoint Bresenham Algorithm

——procedure

在 $0 \leq k \leq 1$ 情况下的整数的中点Bresenham算法：

- (1) 输入直线的两端点 $P_0(x_0, y_0)$ 和 $P_1(x_1, y_1)$ 。
- (2) 计算初始值 Δx 、 Δy 、 $d = \Delta x - 2\Delta y$ 、 $x = x_0$ 、 $y = y_0$ 。
- (3) 绘制点 (x, y) 。

判断 d 的符号

若 $d < 0$ ，则 (x, y) 更新为 $(x+1, y+1)$ ， d 更新为 $d + 2\Delta x - 2\Delta y$ ；

否则 (x, y) 更新为 $(x+1, y)$ ， d 更新为 $d - 2\Delta y$ 。

- (4) 当直线没有画完时，重复步骤3，否则结束。

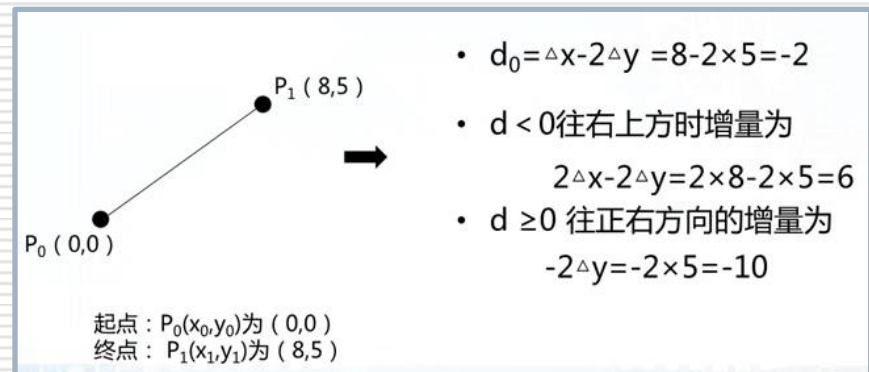
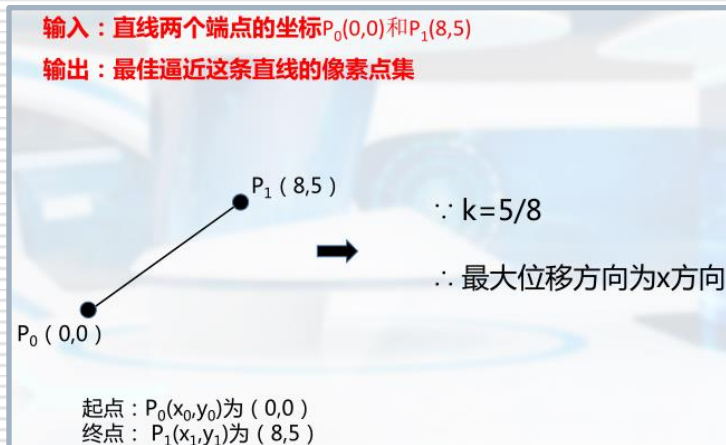
MidPoint Bresenham Algorithm

——pseudocode

```
void Bes_line(int x0, int y0, int x1, int y1, int color)
{
    int dx, dy, d, x, y;
    dx = abs(x0 - x1);    dy = abs(y0 - y1);
    d = dx - 2 * dy;    x = x0; y = y0;
    PutPixel(x, y, color);
    while (x < x1) // 从左端点x0开始向右端点步进
    {
        if (d < 0)
        {
            d += 2 * (dx - dy); y++;
        }
        else
        {
            d -= 2 * dy;
            PutPixel(x, y, color);
            x++;
        }
    }
}
```

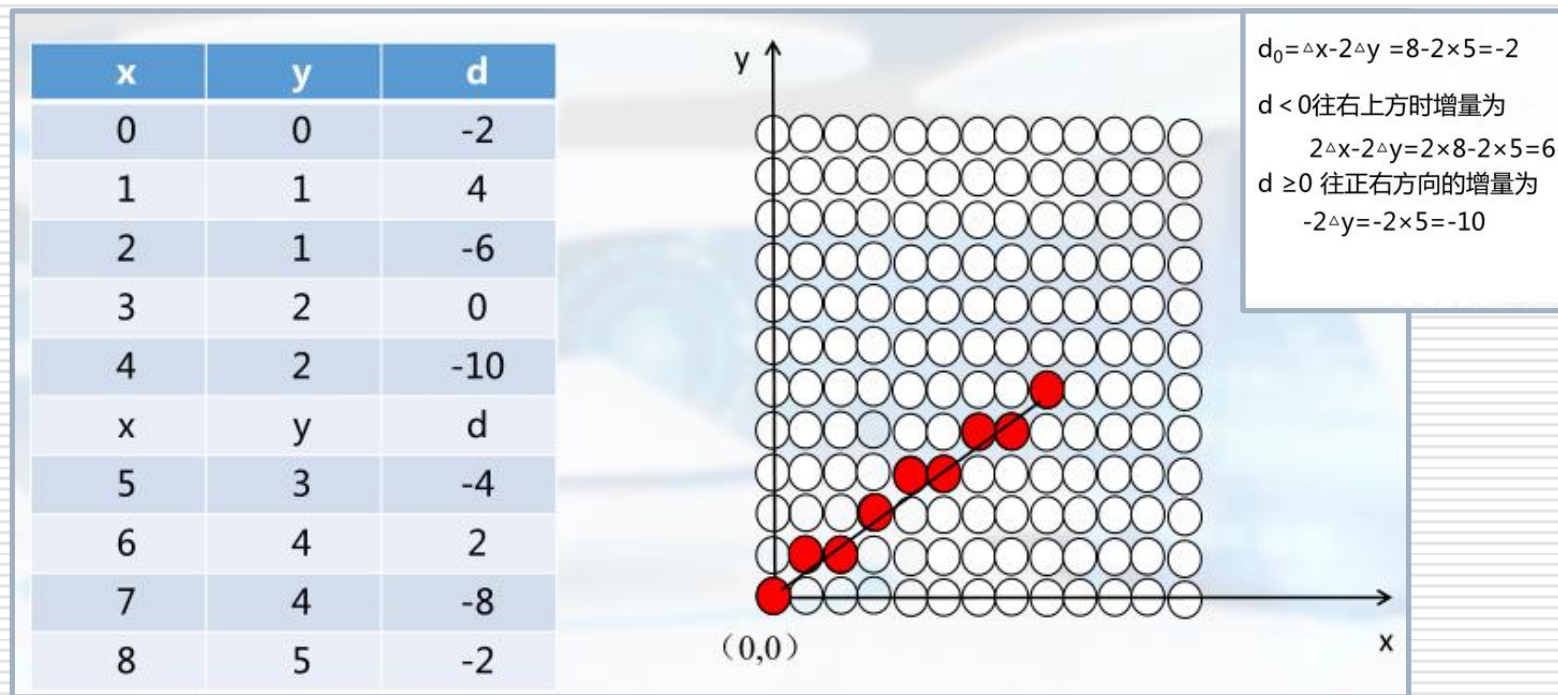
Instance

展示中点Bresenham算法的扫描转换的全过程



Instance

这个表里可以看到误差项的变化过程，也可以看到根据误差项选择点的过程，最终得到右边这条最佳逼近这条直线的像素集合。



中点Bresenham算法(中点画线法)

MidPoint Bresenham Algorithm

□ 优点:

- 只有整数增量运算
- 准确有效
- 对圆和其他曲线同样有效
- 计算机图形学领域使用最广泛的直线扫描转换方法

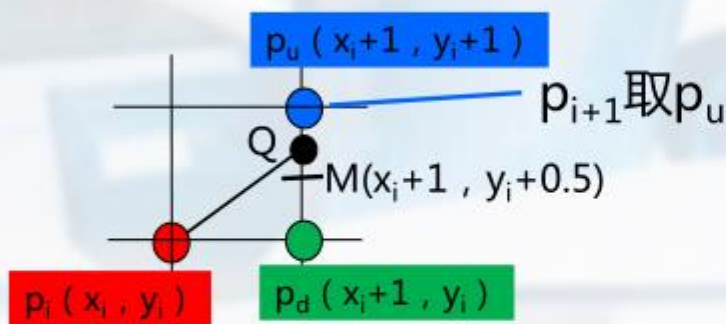
改进的Bresenham算法(Bresenham算法)

Improved Bresenham Algorithm

- 中点Bresenham算法回顾 (假定 $0 \leq k \leq 1$, x 是最大位移方向)

基本原理：

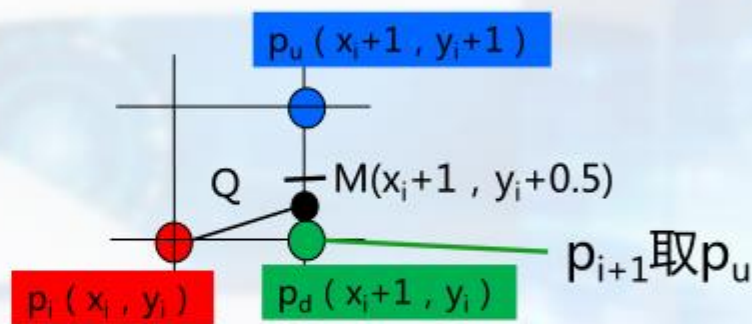
通过中点与直线的位置关系引入误差项 d ，判断误差项的符号来选择候选点



M 在 Q 的下方

$$d = F(M) < 0$$

取 P_u



M 在 Q 的上方

$$d = F(M) \geq 0$$

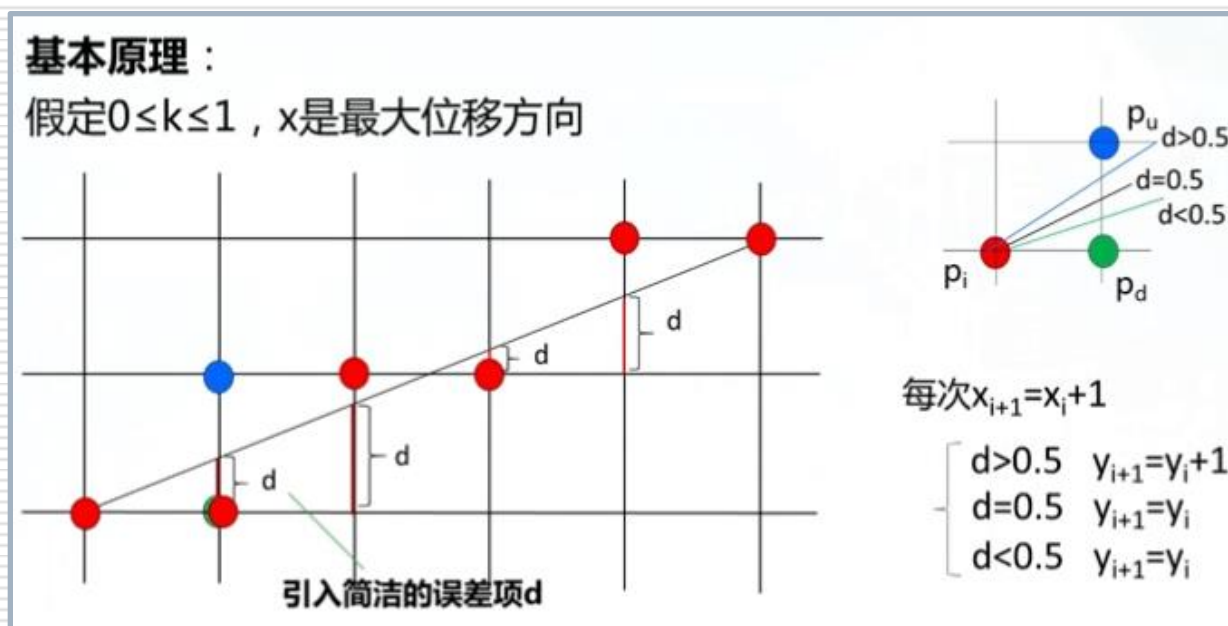
取 P_d

- 该算法很高效，但仍有改进余地，其实我们更直接的定义误差项

改进的Bresenham算法(Bresenham算法)

Improved Bresenham Algorithm

- 更直观的想法

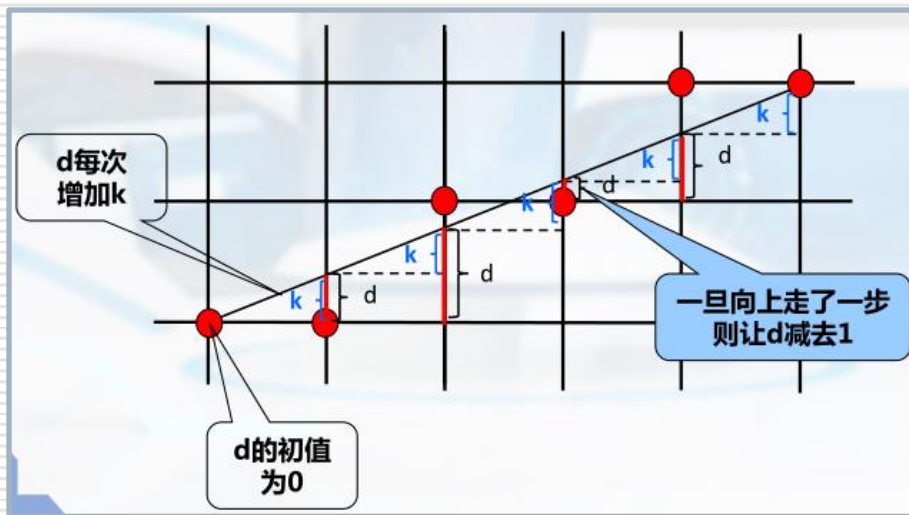


- 理想直线与它垂直方向最近的下光栅点的误差为 d ，通过 d 的值来判断选择哪个候选点。
- 如图所示，也即计算直线与各垂直网格线的交点，然后确定该列像素中与此交点最近的像素，即所选候选点。

改进的Bresenham算法

Improved Bresenham Algorithm

- 寻找d的变换规律



误差项的计算

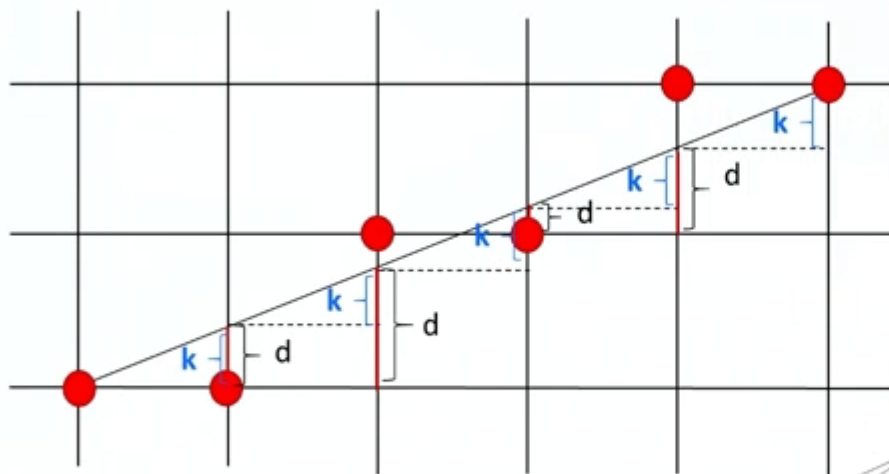
- $d_{初} = 0$,
- 每走一步: $d = d + k$
- 一旦y方向上走了一步, $d = d - 1$

$$\begin{cases} x_{i+1} = x_i + 1 \\ y_{i+1} = \begin{cases} y_i + 1 & (d > 0.5) \\ y_i & (d \leq 0.5) \end{cases} \end{cases}$$

改进的Bresenham算法

Improved Bresenham Algorithm

完备的算法



d 的初值：

$$d_0 = 0$$

d 的变换及如何取点：

$$d = d + k$$

浮点运算

$$x_{i+1} = x_i + 1$$

$$d > 0.5 \quad y_{i+1} = y_i + 1 \quad \text{此时 } d = d - 1$$

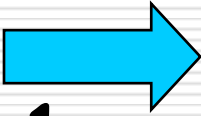
$$d \leq 0.5 \quad y_{i+1} = y_i$$

让人烦恼
的比较

改进1: 令 $e=d-0.5$

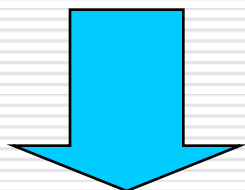
$$\begin{cases} x_{i+1} = x_i + 1 \\ y_{i+1} = \begin{cases} y_i + 1 & (d > 0.5) \\ y_i & (d \leq 0.5) \end{cases} \end{cases} \longrightarrow \begin{cases} x_{i+1} = x_i + 1 \\ y_{i+1} = \begin{cases} y_i + 1 & (e > 0) \\ y_i & (e \leq 0) \end{cases} \end{cases}$$

误差项的计算

- $d_{\text{初}}=0$,
 - 每走一步: $d=d+k$
 - if($d>0.5$)then $d=d-1$
- 
- $e_{\text{初}}=-0.5$,
 - 每走一步有 $e=e+k$ 。
 - if ($e>0$) then $e=e-1$

改进2：用 $E=2e\Delta x$ 来替换 e ,消除浮点数

- $e_{\text{初}} = -0.5$
- 每走一步有 $e = e + k$
- if ($e > 0$) then $e = e - 1$



- $E_{\text{初}} = -0.5 * 2\Delta x = -\Delta x$
- 每走一步有 $E = (e + k) * 2\Delta x = E + 2\Delta y$
- if ($E > 0$) then $E = (e - 1) * 2\Delta x = E - 2\Delta x$

Improved Bresenham Algorithm

——procedure

1. 输入直线的两 endpoint $P_0(x_0, y_0)$ 和 $P_1(x_1, y_1)$ 。
2. 计算初始值 Δx 、 Δy 、 $e = -\Delta x$ 、 $x = x_0$ 、 $y = y_0$ 。
3. 绘制点 (x, y) 。
4. e 更新为 $e + 2\Delta y$ ，判断 e 的符号。
若 $e > 0$ ，则 (x, y) 更新为 $(x + 1, y + 1)$ ，同时将 e 更新为 $e - 2\Delta x$ ；
否则 (x, y) 更新为 $(x + 1, y)$ 。
5. 当直线没有画完时，重复步骤3和4。否则结束。

Improved Bresenham Algorithm

——pseudocode

- 该算法巧妙之处在于：采用增量计算，对于每一列，只要检查一个误差项的符号就可以确定这一列的像素点是哪一个。
- 不依赖于直线方程。

改进的Bresenham画线算法程序：

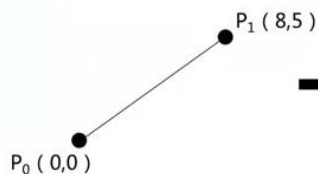
```
void InterBresenhamline (int x0,int y0,int x1, int y1,int color)
{ dx = x1-x0; dy = y1- y0; e=-dx; x=x0; y=y0;
  for (i=0; i<= dx; i++)
    {Putpixel (x, y, color); x++; e=e+2*dy;
      if (e>= 0) { y++; e=e-2*dx;}
    }
}
```

Instance

展示改进Bresenham算法的扫描转换的全过程

输入：直线两个端点的坐标 $P_0(0,0)$ 和 $P_1(8,5)$

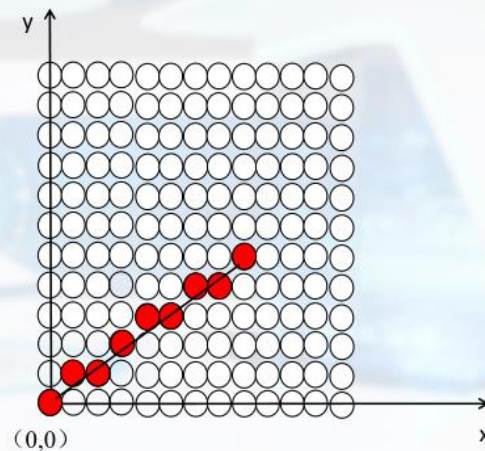
输出：最佳逼近这条直线的像素点集



起点： $P_0(x_0, y_0)$ 为 $(0,0)$
终点： $P_1(x_1, y_1)$ 为 $(8,5)$

- $e_0 = -\Delta x = -8$
- e 增量为 $2\Delta y = 2 \times 5 = 10$
- 如果 $e > 0$ 是 y 向上走一步
 $e = e - 2\Delta x$
其中 $-2\Delta x = -16$

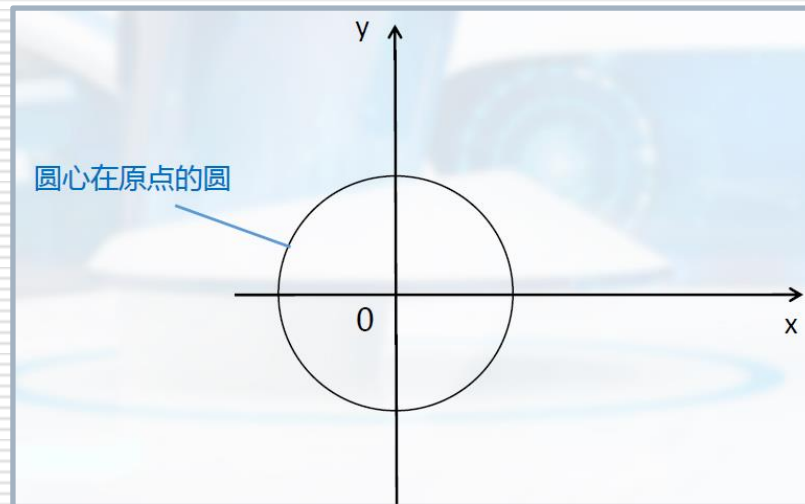
x	y	e	$e + 2\Delta y$
0	0	-8	2
1	1	-14	-4
2	1	-4	6
3	2	-10	0
4	2	0	10
5	3	-6	4
6	4	-12	-2
7	4	-2	8
8	5	-8	2
9	6	-14	



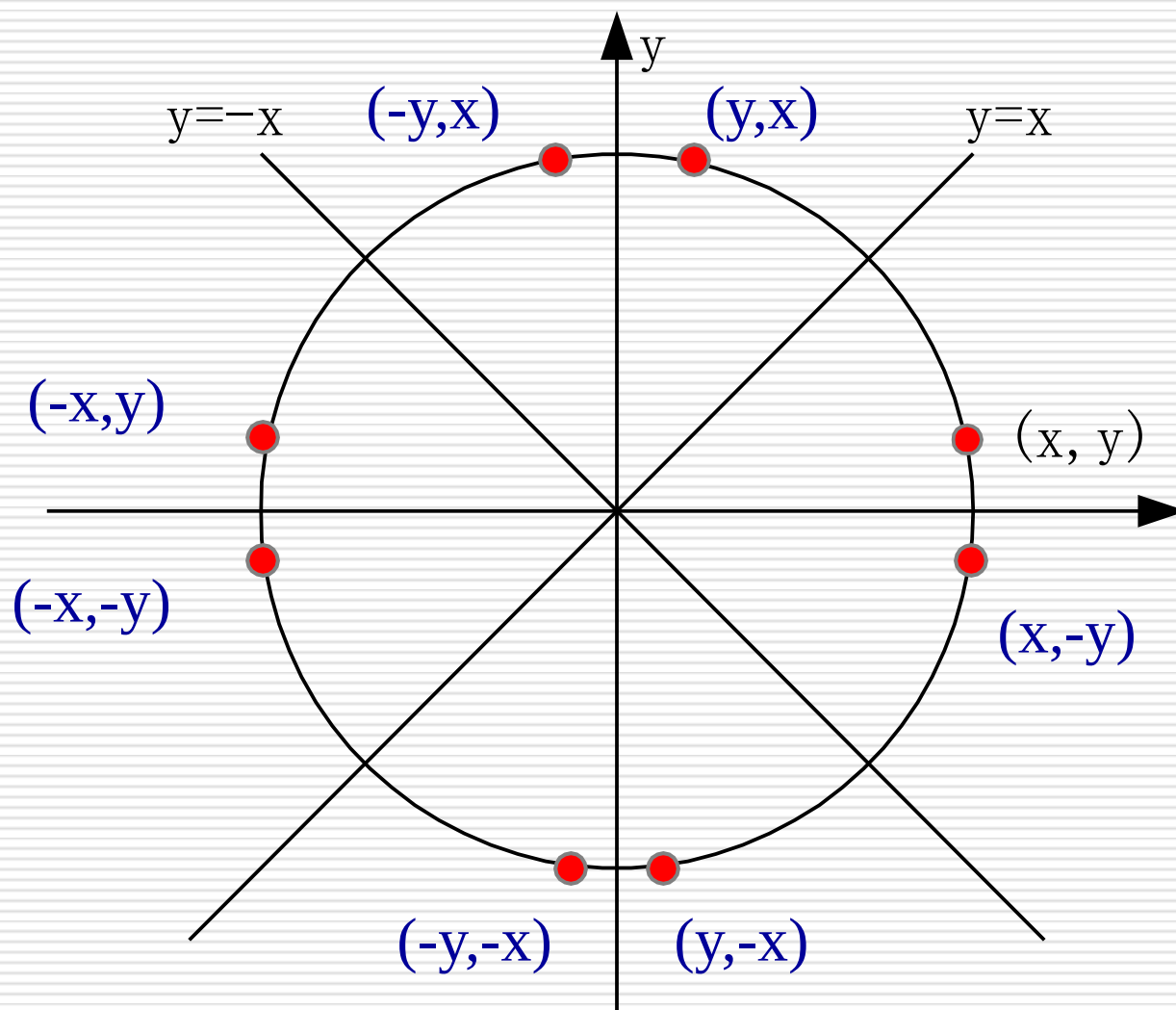
圆的扫描转换 (Circle's generation)

- 简化问题：只考虑圆心在原点的圆
- 解决的问题：绘出圆心在原点，半径为整数 R 的圆

$$x^2 + y^2 = R^2。$$



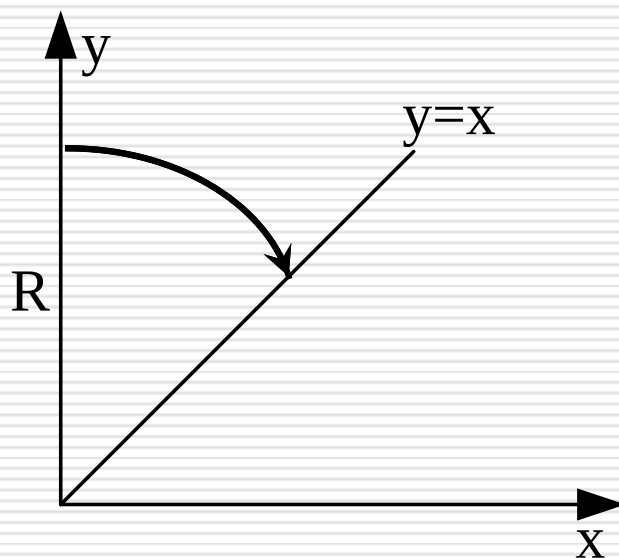
八分法画圆的由来：只需要画出一个八分之一段圆弧即可



在生成封闭的圆周时,只要生成**第一象限的1/8**圆弧,其余部分可利用**对称点**得到。

圆的扫描转换 (Circle's generation)

解决问题：从点 $(0, R)$ 到点 $(R/\sqrt{2}, R/\sqrt{2})$ 顺时针地确定最佳逼近与该圆弧的像素序列。



图： 1/8圆弧

圆的扫描转换 (Circle's generation)

- 简单方程生成圆弧
- 中点Bresenham算法

简单方程产生圆弧

算法原理：利用其函数方程，直接离散计算，两种直接离散的方法如下：

圆的隐函数方程为： $x^2 + y^2 = R^2$

$$x_{i+1} = x_i + 1 \quad x \in [0, R/\sqrt{2}]$$

$$y_{i+1} = \text{round}(\sqrt{R^2 - x_{i+1}^2})$$

- 在第一象限的1/8圆弧中，**x**方向是最大位移方向，**x**方向每次走一步，**y**方向如何走步需要靠公式来计算。

简单方程产生圆弧

圆的极坐标方程为:

$$x = R \cos \theta$$

$$y = R \sin \theta$$

$$\theta_{i+1} = \theta_i + \Delta\theta \quad (\Delta\theta \text{ 为一固定角度步长})$$

$$x_{i+1} = \text{round}(R \cos \theta_{i+1})$$

$$y_{i+1} = \text{round}(R \sin \theta_{i+1})$$

缺点: 乘方、开根, 取整, 三角函数运算, 浮点运算,
乘法运算, 计算量大

中点Bresenham画圆

Midpoint Circle Algorithm

给定圆心在原点，半径为整数**R**的圆，其方程为

$$x^2 + y^2 = R^2$$

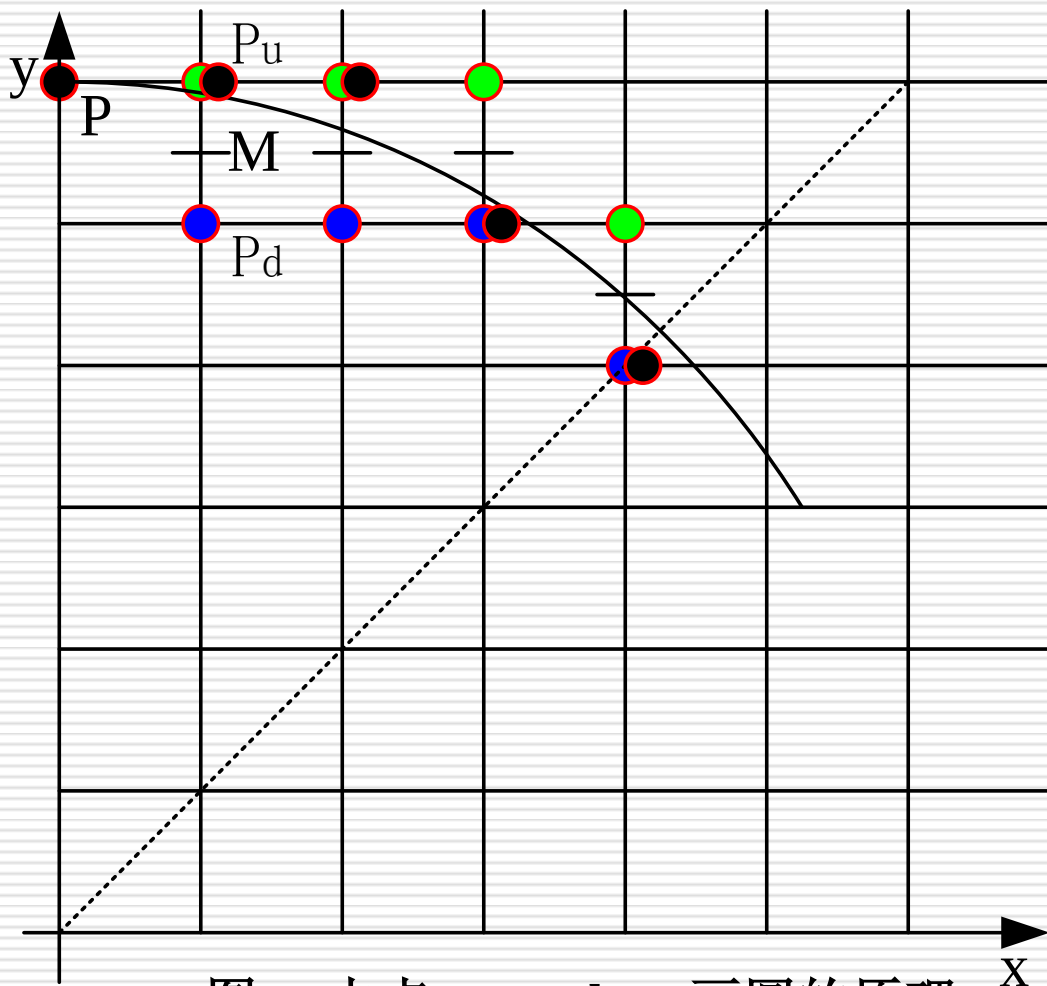
构造函数 **$F(x,y)=x^2+y^2-R^2$** 。

- 对于圆上的点，有 **$F(x,y)=0$** ；
- 对于圆外的点， **$F(x,y)>0$** ；
- 而对于圆内的点， **$F(x,y)<0$** 。

利用圆的对称性，只需讨论1/8圆

与中点画线算法一样，构造如下判别式：

$$d = F(x_M, y_M) = F(x_i + 1, y_i - 0.5) = (x_i + 1)^2 + (y_i - 0.5)^2 - R^2$$



如果 $d < 0$ M在圆内，取哪个点？

取 p_u ！

如果 $d > 0$ M在圆外，取哪个点？

取 p_d ！

如果 $d = 0$ M在圆上，取哪个点？

任取一个，约定取 p_u ！

图：中点Bresenham画圆的原理

中点Bresenham画圆

Midpoint Circle Algorithm

构造判别式:

$$d = F(x_M, y_M) = F(x_i + 1, y_i - 0.5) = (x_i + 1)^2 + (y_i - 0.5)^2 - R^2$$

- 当 $d \leq 0$ 时, 下一点取 $P_u(x_i + 1, y_i)$;
- 当 $d > 0$ 时, 下一点取 $P_d(x_i + 1, y_i - 1)$ 。

误差项的递推 ($d \leq 0$)

$$\begin{aligned} d_1 &= F(x_M, y_M) = F(x_i+1, y_i-0.5) \\ &= (x_i+1)^2 + (y_i-0.5)^2 - R^2 \end{aligned}$$

$$\begin{aligned} d_2 &= F(x_i+2, y_i-0.5) \\ &= (x_i+2)^2 + (y_i-0.5)^2 - R^2 \end{aligned}$$

$$\begin{aligned} d_2 &= (x_i+1+1)^2 + (y_i-0.5)^2 - R^2 \\ &= (x_i+1)^2 + 2x_i + 3 + (y_i-0.5)^2 - R^2 \\ &= d_1 + 2x_i + 3 \end{aligned}$$

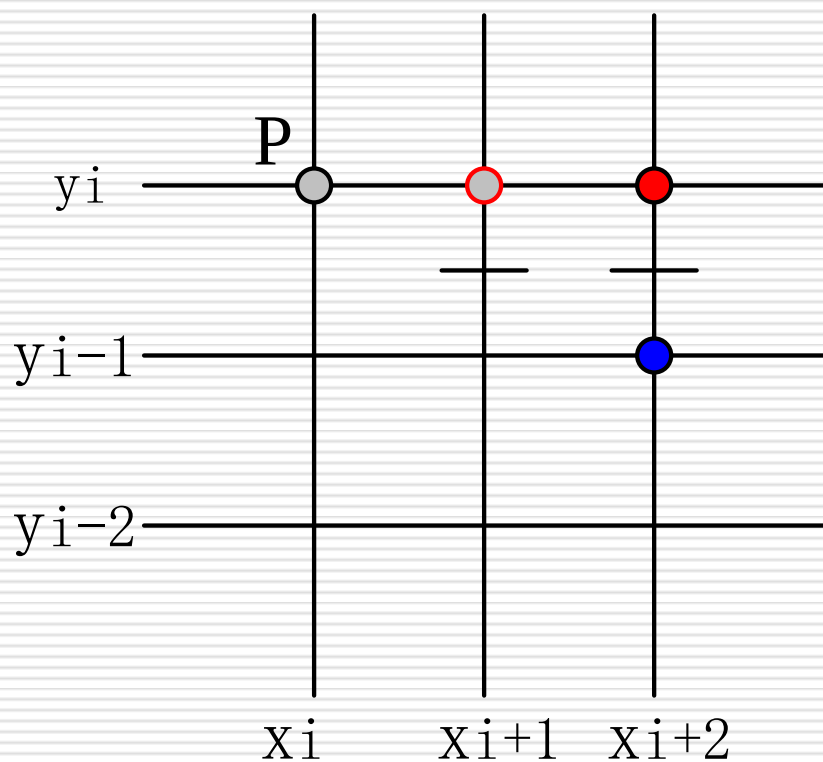


图: $d \leq 0$ 的情况

误差项的递推 ($d > 0$)

$$d_1 = F(x_M, y_M) = F(x_i + 1, y_i - 0.5)$$

$$= (x_i + 1)^2 + (y_i - 0.5)^2 - R^2$$

$$d_2 = F(x_i + 2, y_i - 1.5)$$

$$= (x_i + 2)^2 + (y_i - 1.5)^2 - R^2$$

$$d_2 = (x_i + 1 + 1)^2 + (y_i - 0.5 - 1)^2 - R^2$$

$$= (x_i + 1)^2 + 2x_i + 3 + (y_i - 0.5)^2 - 2(y_i - 0.5) + 1 - R^2$$

$$= (x_i + 1)^2 + (y_i - 0.5)^2 + 2(x_i - y_i) + 5$$

$$= d_1 + 2(x_i - y_i) + 5$$

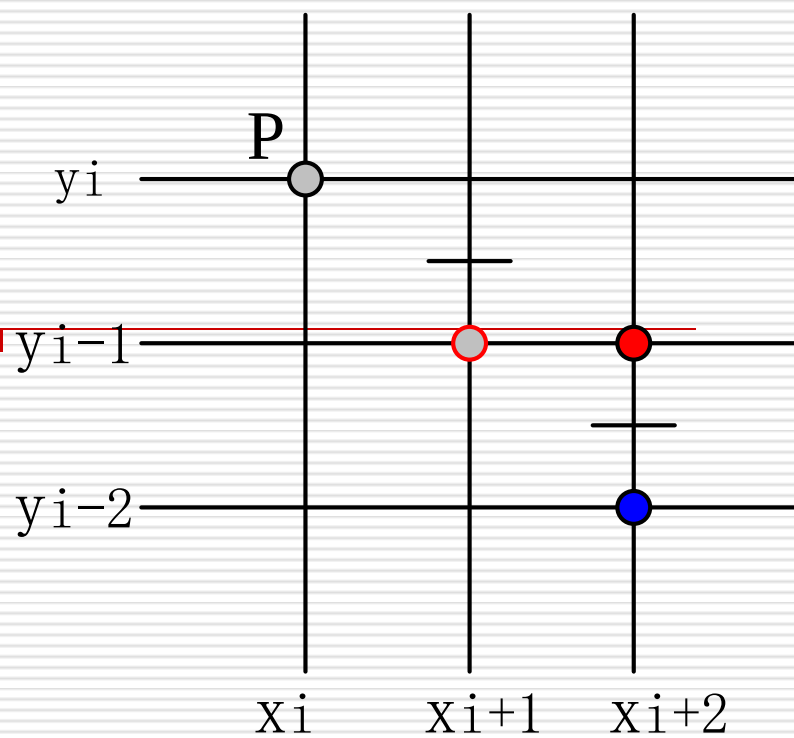


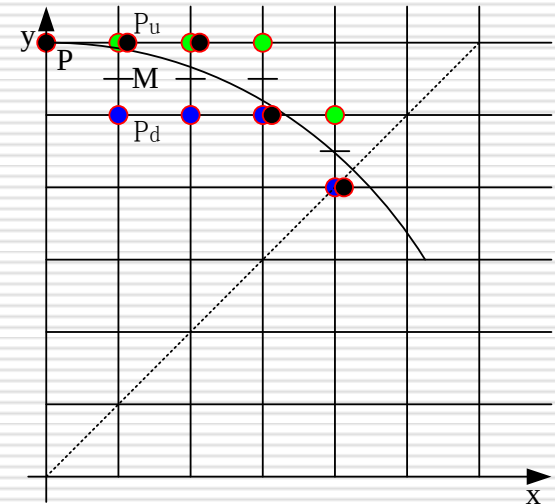
图: $d > 0$ 的情况

中点Bresenham画圆

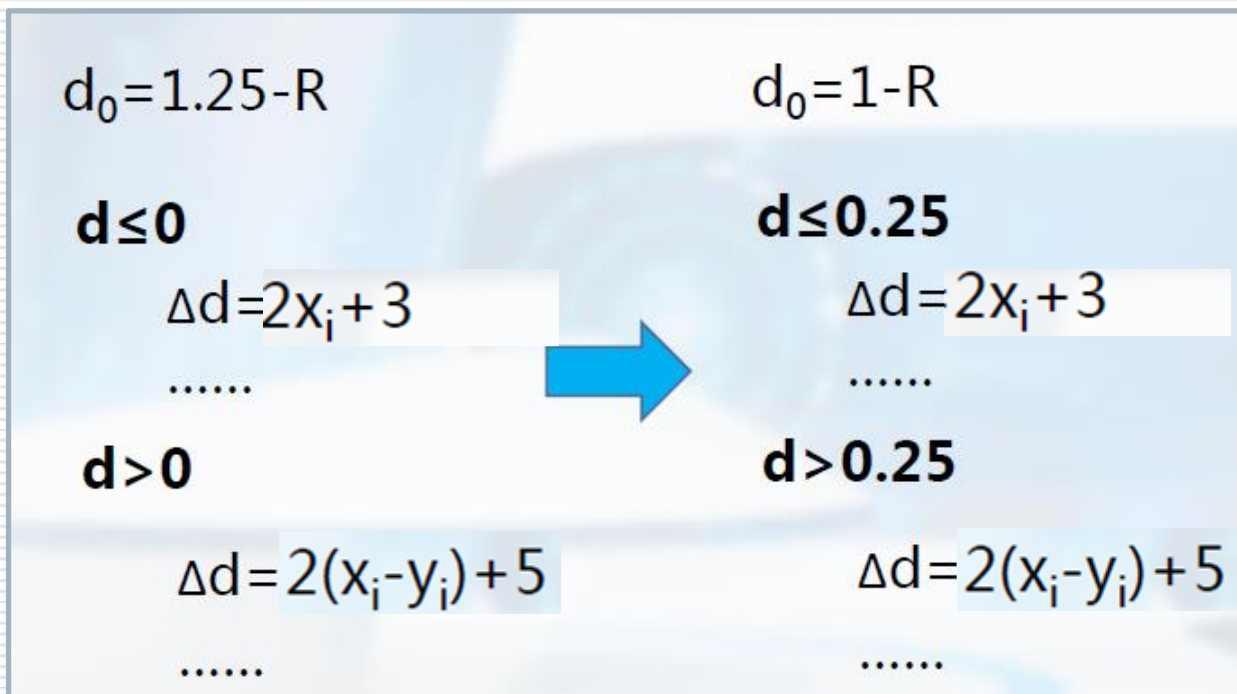
Midpoint Circle Algorithm

判别式的初始值，要生成的八分圆的第一个像素是 $(0, R)$ 。

$$\begin{aligned}d_0 &= F(x_0 + 1, y_0 - 0.5) \\&= F(1, R - 0.5) \\&= 1 + (R - 0.5)^2 - R^2 \\&= 1.25 - R\end{aligned}$$



改进：用 **$d-0.25$** 代替 **d** ，把浮点数改成整数，此时有：



$$d \leq -0.25$$

$$d > -0.25$$



$$d < 0$$

$$d \geq 0$$

- d** 的初值，增量也是整数，这时算法只涉及到整数，所以 **d** 与 **-0.25** 的比较根本就没有必要，我们可以把它直接改为 **d** 与 **0** 的比较，只涉及到 **d** 的符号。

中点Bresenham画圆——procedure

1. 输入圆的半径 R 。
2. 计算初始值 $d=1-R$ 、 $x=0$ 、 $y=R$ 。
3. 绘制点 (x,y) 及其在八分圆中的另外七个对称点。
4. 判断 d 的符号。
 - 若 $d < 0$ ，则先将 d 更新为 $d+2x+3$ ，再将 (x,y) 更新为 $(x+1,y)$ 。
 - 否则先将 d 更新为 $d+2(x-y)+5$ ，再将 (x,y) 更新为 $(x+1,y-1)$ 。
5. 当 $x < y$ 时，重复步骤3和4。否则结束。

中点Bresenham画圆——procedure

程序如下（完全用整数实现）：

```
MidpointCircle(r,color)
Int r, color;
{
    int x,y,d;
    x = 0; y = r; d = 1-r;
    putpixel(x,y,color);
```

```
    while( x < y)
    { if (d < 0)
      { d += 2*x+3; x++; }
      else
      { d += 2*(x-y)+5;
        x++; y--; }
      putpixel(x,y,color);
    }
}
```

上述算法能否再改进呢？

（思路：对增量再进行增量计算）

- 注意到 d 的增量是 x,y 的线性函数，
- 每当 x 递增1，则 d 的增量递增 $\Delta x=2$
- 每当 y 递减1，则 d 的增量递增 $\Delta y=2$
- Δx 初始值=3； Δy 初始值=-2 r +2

中点Bresenham画圆 -- pseudocode

```
x=0; y=r; d=1-r;
```

```
.....
```

```
if(d<0)
```

```
{ d+ = 2*x+3; x++ ;}
```

```
else
```

```
{d+ = 2*(x-y) + 5; x++;y--; }
```

```
x=0;y=r;d=1-r;deltax=3;delty=-2*r+2
```

```
.....
```

```
if(d<0)
```

```
{ d+ = deltax; x++; deltax+=2 ;}
```

```
else
```

```
{d+ = deltax+delty; x++;y--;
```

```
deltax+=2; delty+=2 }
```

```
MidpointCircle(int r, int color)
```

```
{
```

```
    int x,y;
```

```
    int d;
```

```
    x=0; y=r; d=1-r; deltax =3; delty =-2*r+2;
```

```
    while(x<y)
```

```
    {
```

```
        drawpixel(x,y,color);
```

```
        if(d<0)
```

```
            { d+ = deltax; x++; deltax +=2 }
```

```
        else
```

```
            {d+ = deltax + delty; x++;y--;
```

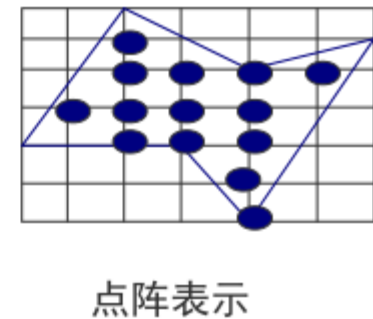
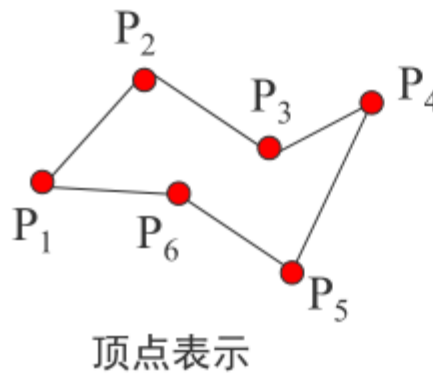
```
            deltax +=2; delty+=2; }
```

```
    }
```

多边形的扫描转换与区域填充

Polygon's scan conversion and Region Filling

- 通常来说，多边形主要有以下两种典型表示方法：
- 顶点表示用**多边形的顶点序列**来刻画多边形。这种表示直观、几何意义强、占内存少，易于进行几何变换。但没有明确指出哪些像素在多边形内，故不能直接用于面着色。
 - **点阵表示**是用位于**多边形内的像素的集合**来刻画多边形。这种表示丢失了许多几何信息，但却是光栅显示系统显示时（面着色）所需的表示形式。



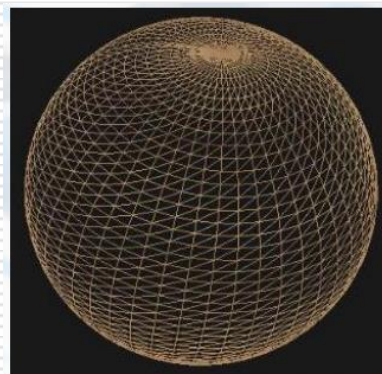
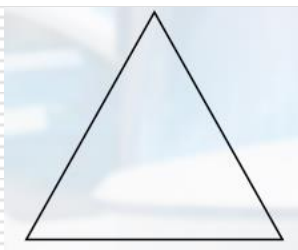
多边形的扫描转换与区域填充

Polygon's scan conversion and Region Filling

- **多边形的扫描转换**：把多边形的顶点表示转化为点阵表示。主要是通过确定穿越区域的扫描线的覆盖区间来填充。
- **区域填充**是从给定的位置开始涂描直到指定的边界条件为止。

多边形的扫描转换 (Polygon's scan conversion)

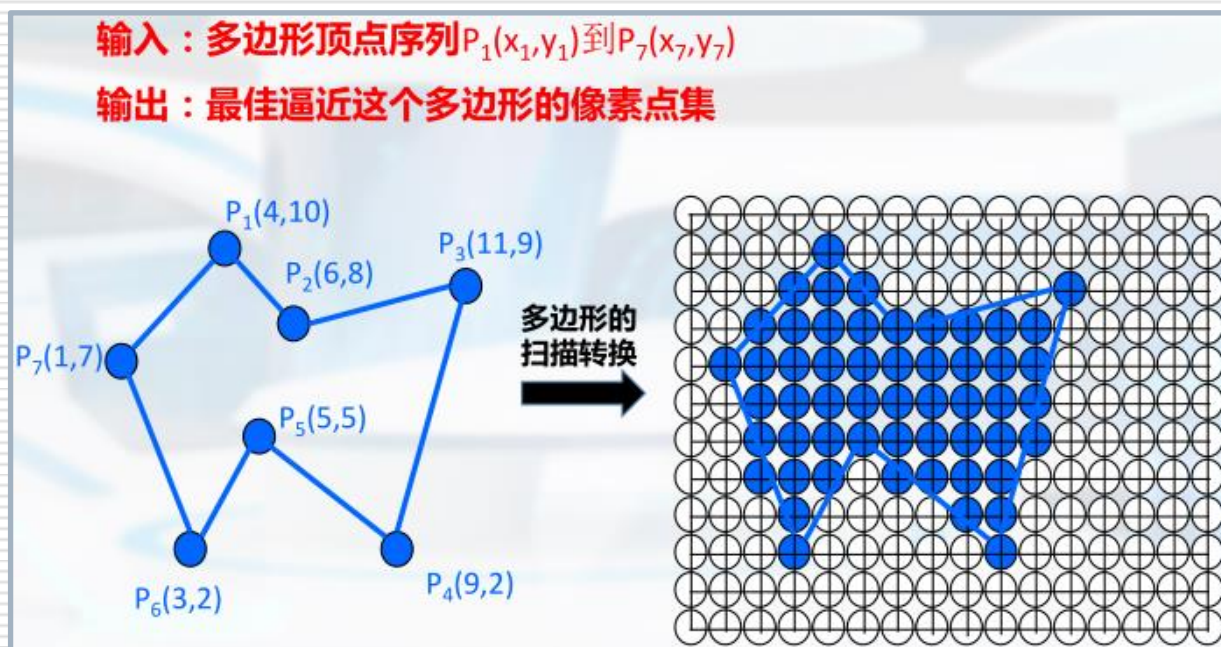
- 图形的扫描转换中，很多时候需要表示一个区域。
- 对区域内颜色进行处理是非常重要的。



多边形的扫描转换

(Polygon's scan conversion)

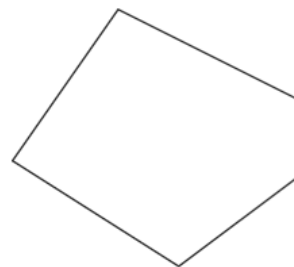
- **扫描转换多边形**：从多边形的顶点信息出发，求出位于其内部的各个象素，并将其颜色值写入帧缓存中相应单元的过程。即**把多边形的顶点表示转换为点阵表示**。（大多数图形应用系统采用顶点序列表示多边形，但顶点表示又不能直接用于显示）



多边形的分类

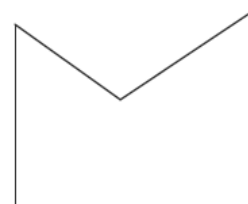
(1) 凸多边形

任意两顶点间的连线均在多边形内



(2) 凹多边形

任意两顶点间的连线有不在在多边形内



(3) 含内环的多边形

多边形内包含多边形



多边形的扫描转换

(Polygon's scan conversion)

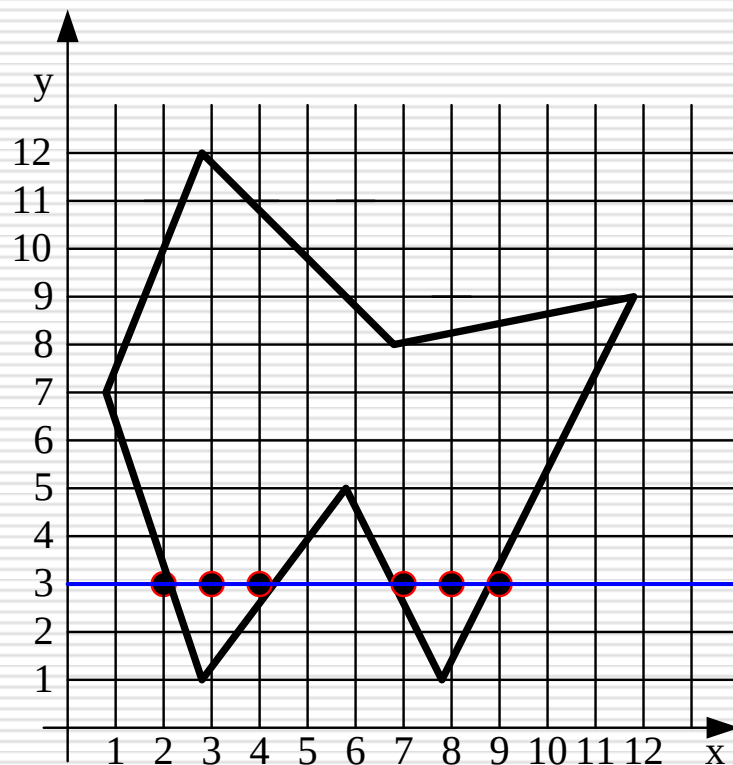
已知多边形的顶点序列，如何找到多边形内部的点，即把多边形内部填上颜色？

- X—扫描线算法
- 改进的**有效边表算法**(Y向连贯性算法)
- 边标记算法(讲解思想)

X-扫描线算法

scan-line polygon algorithm--Idea

- 基本思想：按扫描线顺序，计算扫描线与多边形的相交区间，再用要求的颜色显示这些区间的所有像素。



图： x-扫描线算法填充多边形

X-扫描线算法

scan-line polygon algorithm--procedure

1. 确定多边形所占有的最大扫描线数，得到多边形顶点的最小和最大 y 值（ $y_{\min}$ 和 $y_{\max}$ ）。

2. 从 $y=y_{\min}$ 到 $y=y_{\max}$ ，每次用一条扫描线进行填充。

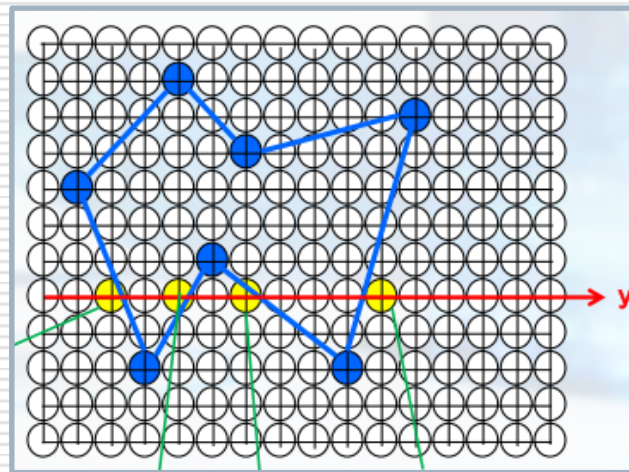
3. 对一条扫描线填充的过程可分为四个步骤：

求交：计算扫描线与多边形各边的交点。

排序：把所有交点按 x 值递增顺序进行排序。

交点配对：每对交点代表扫描线与多边形的一个相交区间。

区间填色：把这些相交区间内的像素置成填充色。



填充过程实例

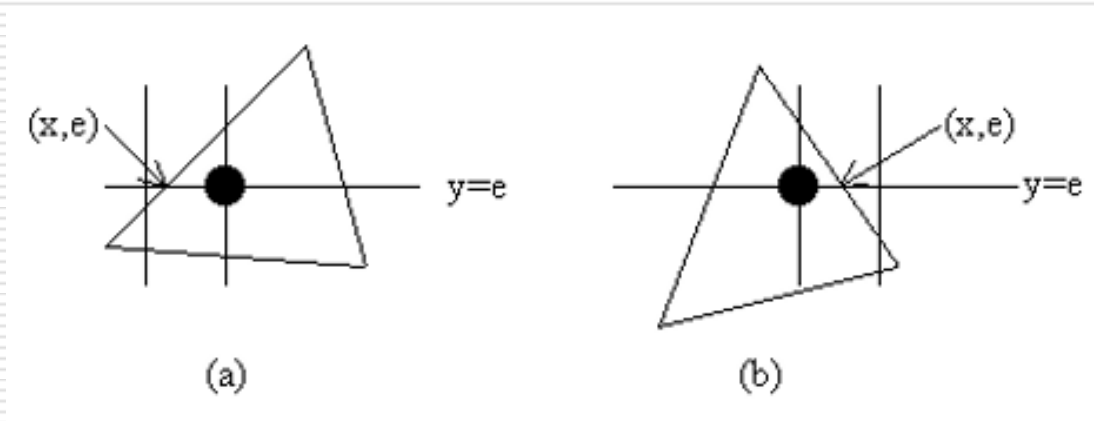
X-扫描线算法——取整规则

通常，为了避免多边形区域的扩大化，需要对交点进行处理，处理规则如下：

- 交点的取整规则：使生成的像素全部位于多边形之内。
（用于直线等图元扫描转换的四舍五入原则可能导致部分像素位于多边形之外，从而不可用）。
- 假定非水平边与扫描线 $y=e$ 相交，交点的横坐标为 x ，规则如下：

□ **规则1**: **X为小数**，即交点落于扫描线上两个相邻像素之间时：

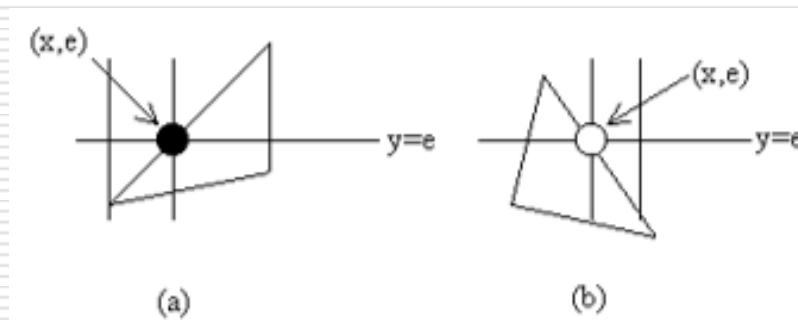
- 交点位于左边界之上，向右取整；
- 交点位于右边界之上，向左取整；



图：取整规则1

X-扫描线算法——取整规则

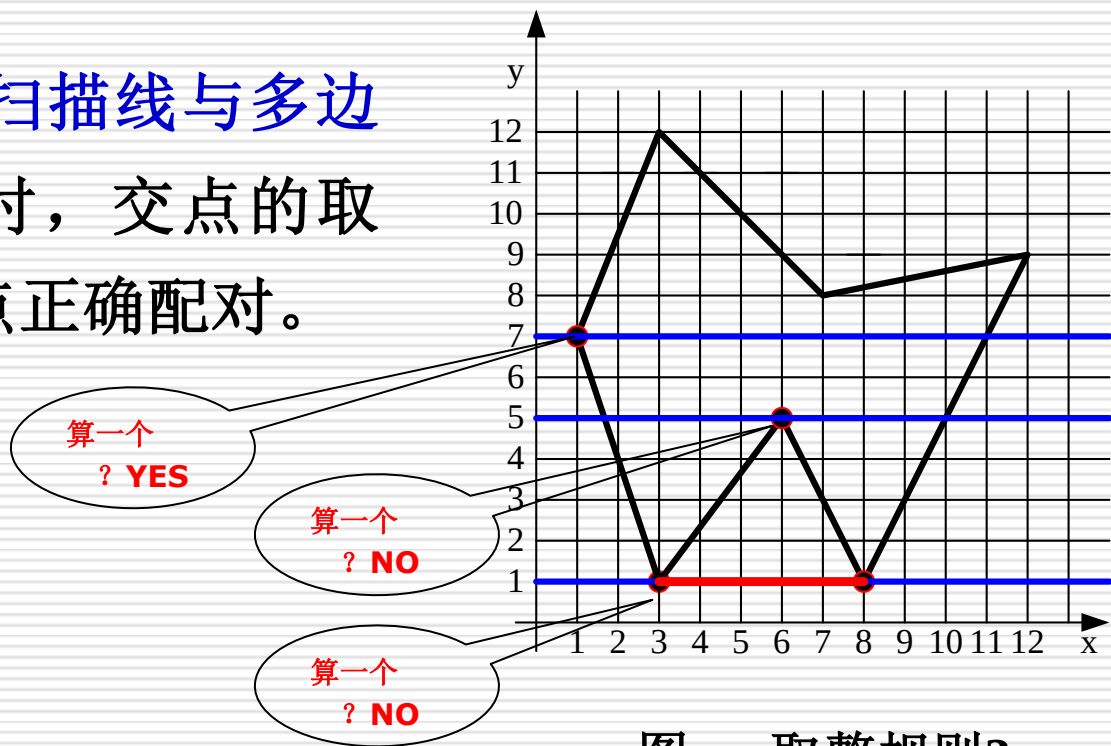
□ 规则2：若 x 是整数，边界上像素的取舍问题，避免填充扩大化。规定落在右边边界上的像素不予填充。(具体实现时，只要对扫描线与多边形的相交区间左闭右开)，类似的有上开下闭原则。



图：取整规则2

X-扫描线算法——交点取舍

□ **规则3**：当扫描线与多边形顶点相交时，交点的取舍，保证交点正确配对。



图： 取整规则3

X-扫描线算法——取整规则

解决方法:

当扫描线与多边形的顶点相交时,

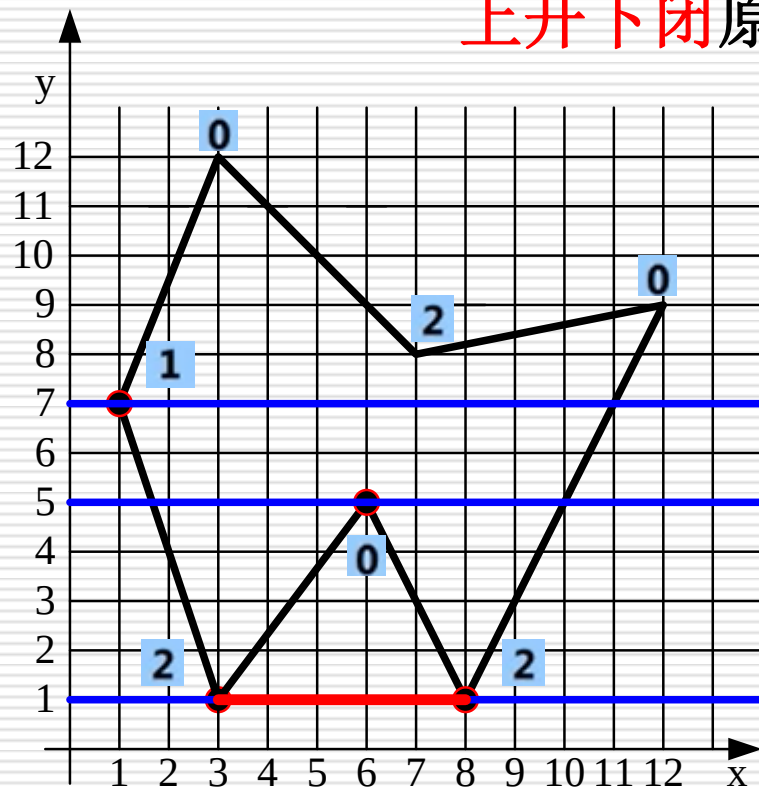
- 若共享顶点的两条边分别落在扫描线的两边, 交点只算一个;
- 若共享顶点的两条边在扫描线的同一边, 这时交点作为零个或两个。(即对多边形的局部最高点和局部最低点进行偶数化处理)

X-扫描线算法——取整规则

实际处理： 只要检查顶点的两条边的另外两个端点的Y值，两个Y值中大于交点Y值的个数是0，1，2，来决定取0，1，2个交点。

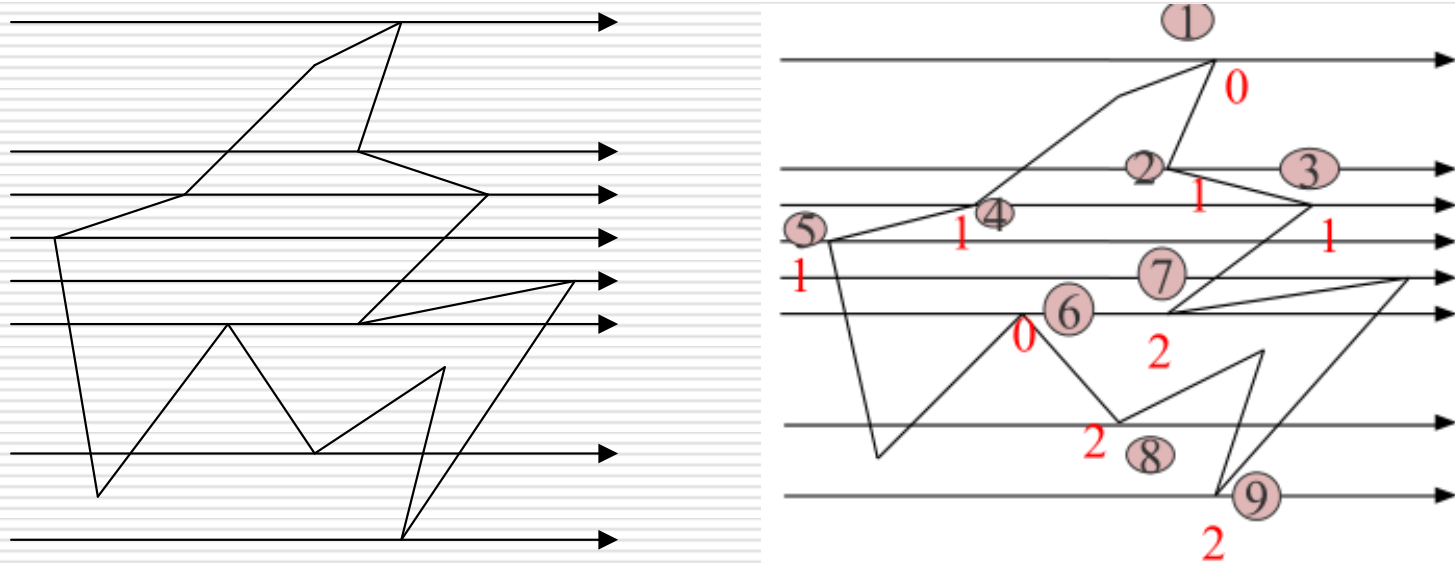
即： 交点个数 = 构成这个顶点的两条边位于扫描线上方的条数。

上开下闭原则



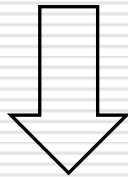
图： 取整规则3

X-扫描线算法——取整规则



图：与扫描线相交的多边形顶点的交点数

✗—扫描线算法效率低：
求交
排序
计算量非常大



改进？

有效边表算法 (Y向连贯性算法)

改进的出发点：

(3) 每次都需要排序吗？

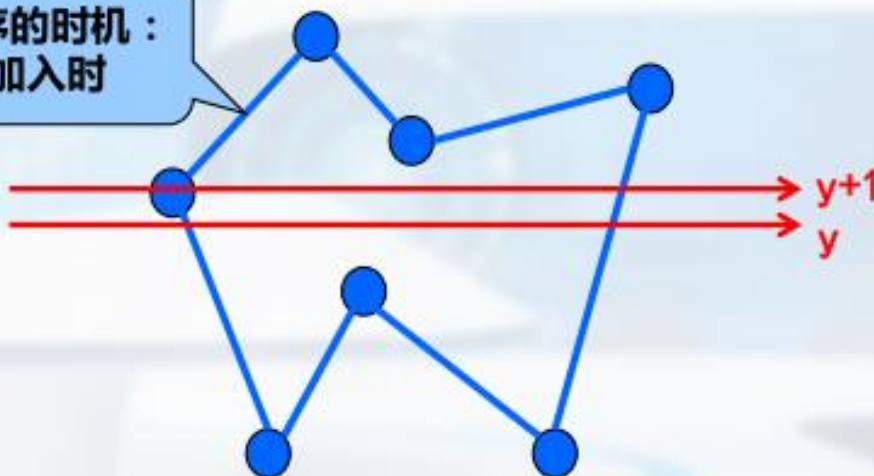
a. 求交

b. 排序

c. 交点配对

d. 区间填色

重新排序的时机：
新边加入时



利用扫描线的连贯性，当前扫描线与各边的交点顺序与下一条扫描线与各边交点的顺序很可能相同或相似。

有效边表算法 (Y向连贯性算法)

改进思想:

连贯性的思想+增量思想

- 利用扫描线的连贯性。
- 利用多边形边的连贯性。

另外, 为了避免复杂运算, 需要构建一套特殊的数据结构:
有效边表+新边表

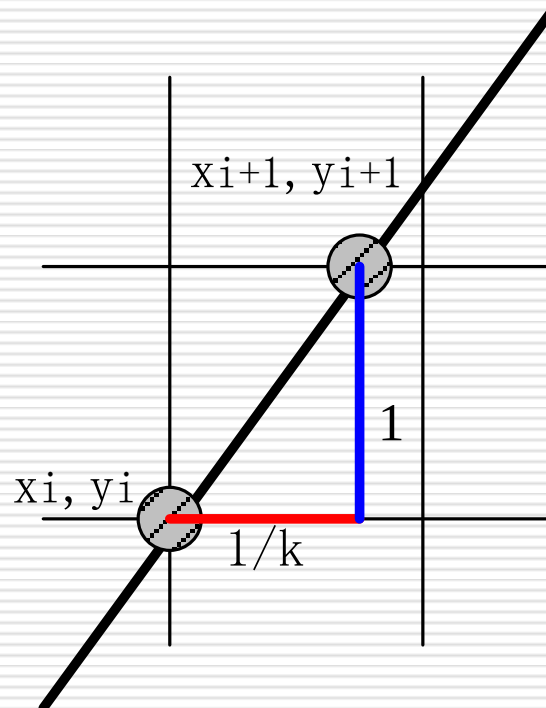


图: 与多边形边界相交的两条连续扫描线交点的相关性

有效边表算法 (Y向连贯性算法)

□ 有效边表 (Active Edge Table, AET) :

把有效边按与扫描线交点 x 坐标递增的顺序存放在一个链表中, 此链表称为有效边表。

有效边 (Active Edge) : 指与当前扫描线相交的多边形的边, 也称为活性边。

□ 有效边表的每个结点:

x	$y_{\max}$	$1/k$	next
-----	------------	-------	------

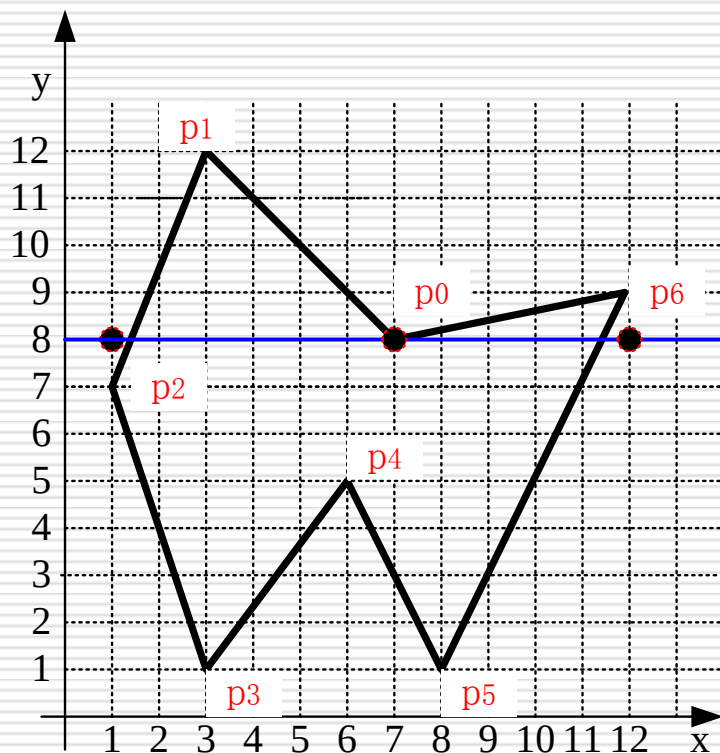
x : 当前扫描线与有效边交点的 x 坐标

$y_{\max}$: 有效边所在的最大扫描线值 $y_{\max}$ (通过它可以知道何时才能“抛弃”该边)

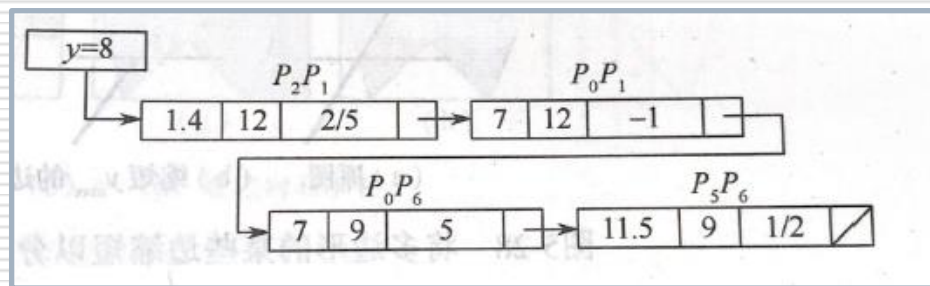
$1/k$: 从当前扫描线到下一条扫描线间 x 的增量

next: 指向下一条有效边的指针

有效边表算法 (Y向连贯性算法)



$y=8$ 这条扫描线的有效边表表示如下:

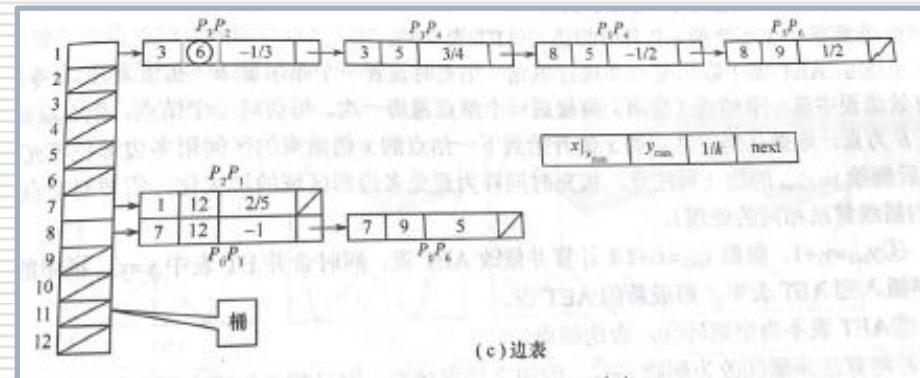


有效边表算法——构造边表

□ 为了方便有效边表的建立与更新，需构造一个**新边表（NET）**（或叫边表，**ET**），用来存放**多边形的边的信息**，分为**4**个步骤：

（1）首先构造一个**纵向链表**，链表的长度为多边形所占有的最大扫描线数，链表的每个结点，称为一个**桶**，则对应多边形覆盖的每一条扫描线。

（2）将**每条边**的信息链入与**该边最小 y 坐标($y_{\min}$)**相对应的桶处。也就是说，若某边的较低端点为 $y_{\min}$ ，则该边就放在相应的 $y=y_{\min}$ 扫描线桶中。存放在**该扫描线第一次出现的边**。



有效边表算法——构造边表

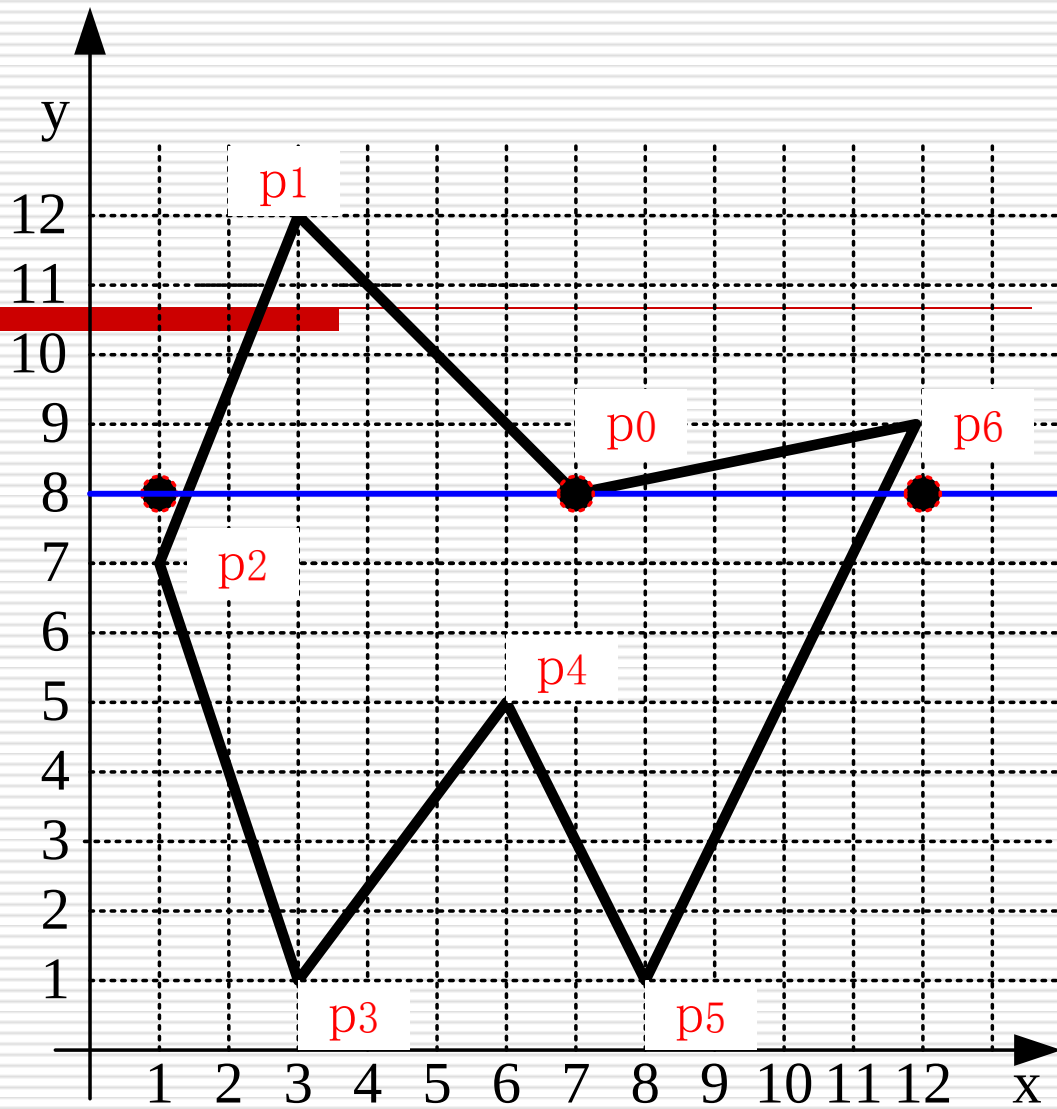
(3) 每条边的数据形成一个结点，内容包括：该扫描线与该边的初始交点 x （即较低端点的 x 值），该边的最大 y 值 $y_{\max}$ ，以及 $1/k$ 。

$x|_{y_{\min}}$ $y_{\max}$ $1/k$ NEXT

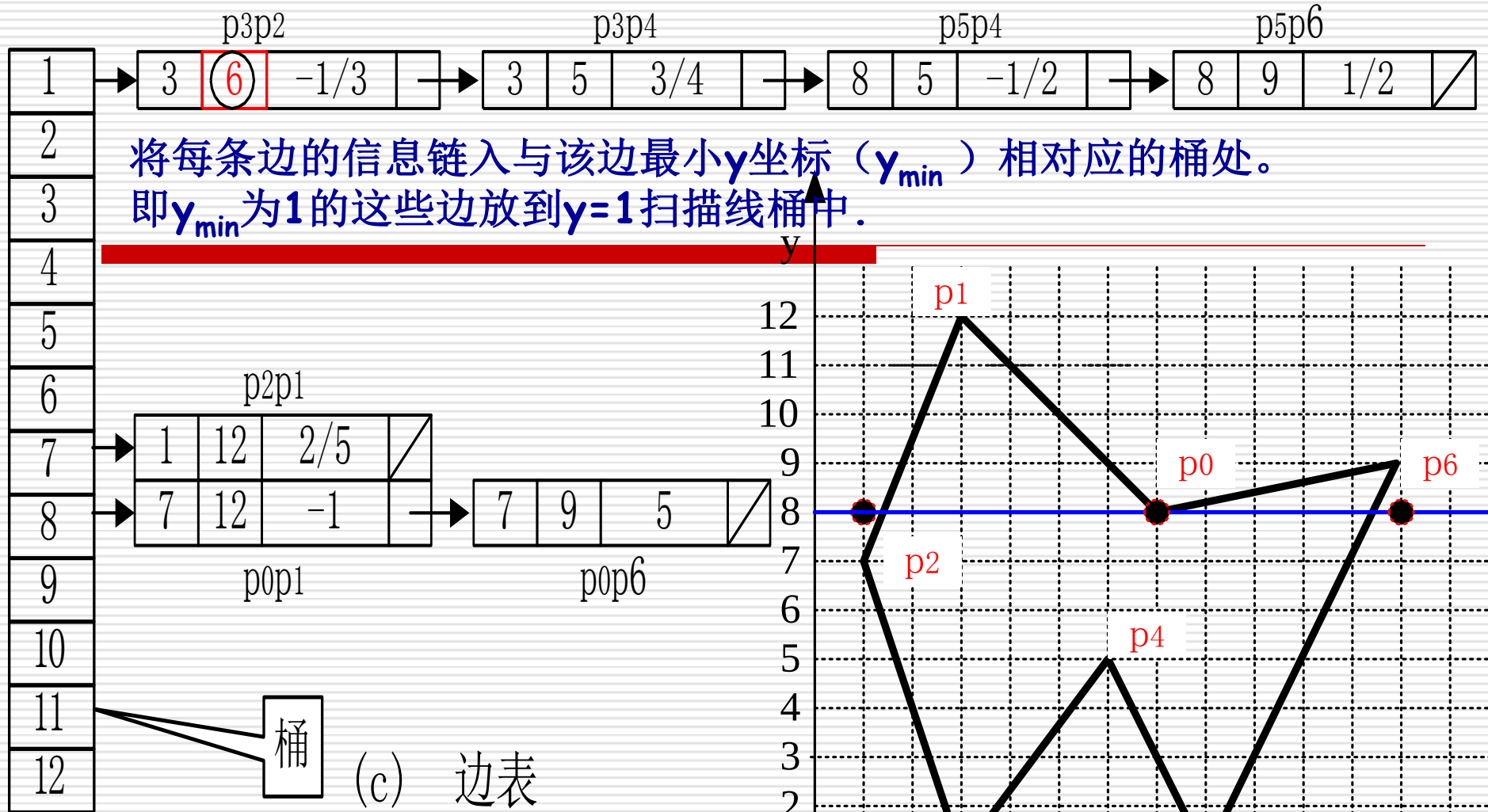
(4) 同一桶中若干条边按 $x|_{y_{\min}}$ 由小到大排序，若 $x|_{y_{\min}}$ 相等，则按照 $1/k$ 由小到大排序。

Example:

p3p2	3	6	$-1/3$	
p3p4	3	5	$3/4$	
p5p4	8	5	$-1/2$	
p5p6	8	9	$1/2$	
p2p1	1	12	$2/5$	
p0p1	7	12	-1	
p0p6	7	9	5	



图： 多边形 $P_0P_1P_2P_3P_4P_5P_6$

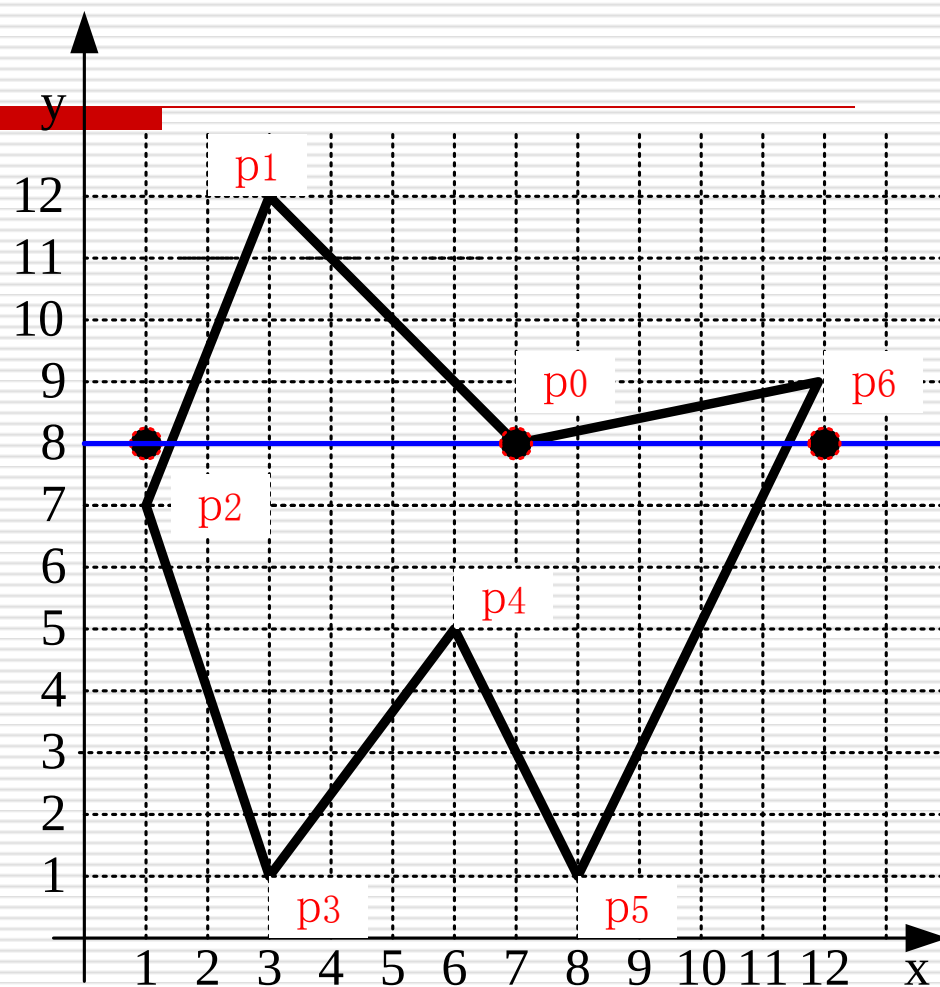


边表ET: 从ET表里就知道多边形的边是从哪里开始的。

图: 多边形 $P_0P_1P_2P_3P_4P_5P_6$

思考：为什么 P_3P_2 这条边对应结点的 $y_{\max}$ 为6，而不是7？

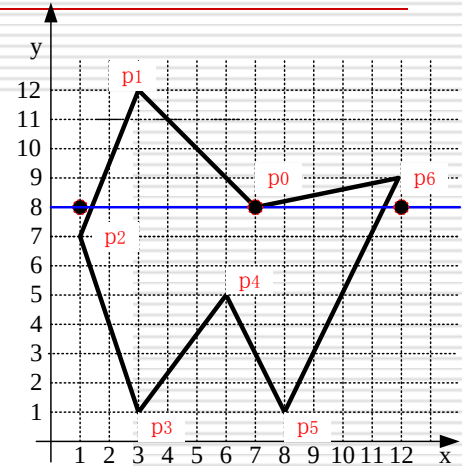
p3p2	3	6	$-1/3$	
------	---	---	--------	--



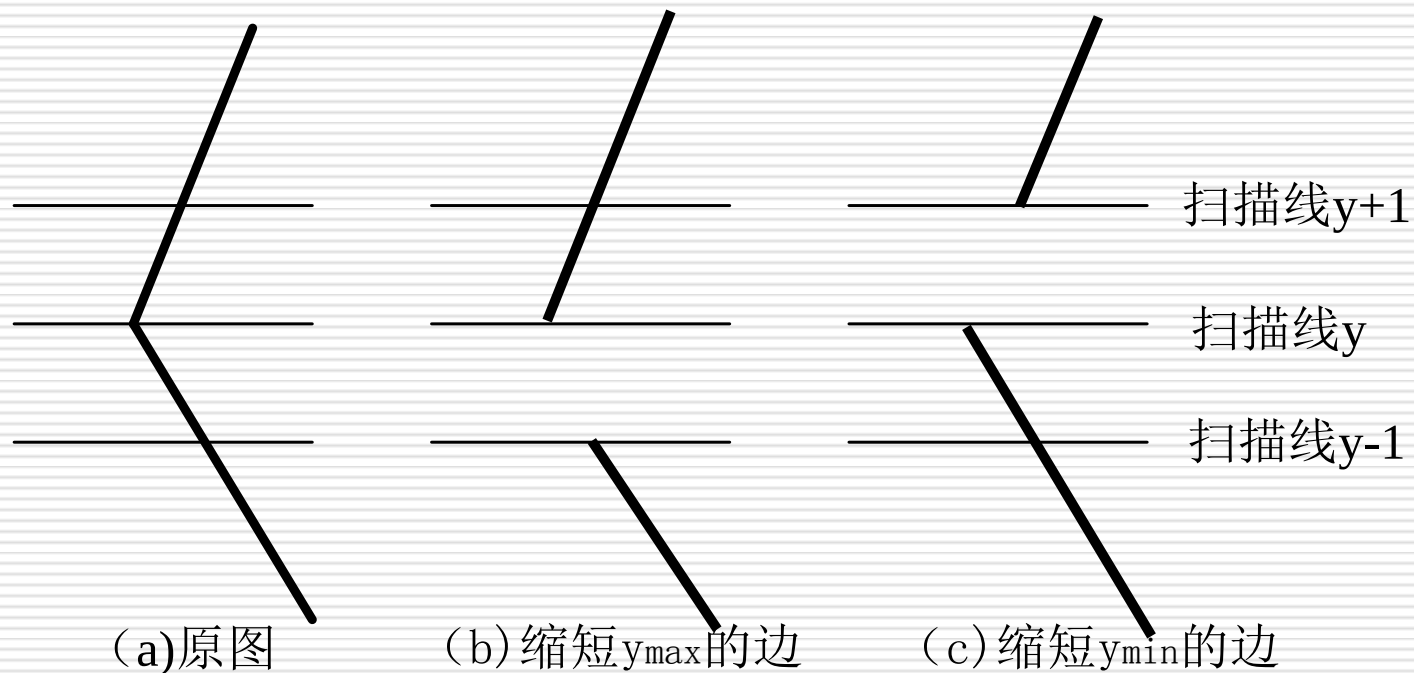
有效边表算法(Y向连贯性算法)—— procedure

◆ 算法步骤

- 1) 初始化: 构造边表**ET**, 有效边表**AET**置空
- 2) 将第一个不空的**ET**表中的边与**AET**表合并
- 3) 由**AET**表中取出交点对进行填充。填充之后删除 $y=y_{\max}$ 的边 //这一步中, 必须对交点为1的情况进行处理, 缩短 $y_{\max}$ 的边, 否则交点个数为奇数, 没法配对!
- 4) $y_{i+1}=y_i+1$, 根据 $x_{i+1}=x_i+1/k$ 计算并修改**AET**表, 同时合并**ET**表中 $y=y_{i+1}$ 桶中的边, 按次序插入到**AET**表中, 形成新的**AET**表
- 5) **AET**表不为空则转步骤3, 否则结束



- 由于配对原则，在顶点相交处，会影响到配对，采用的做法是，在影响配对的情况下进行 $y_{\max}=y_{\max}-1$ 处理，通过此来保证有效边表中的有效边数量为偶数）。
- 解决顶点交点为1的情形如下：(计为0个或2个时本算法不存在问题)



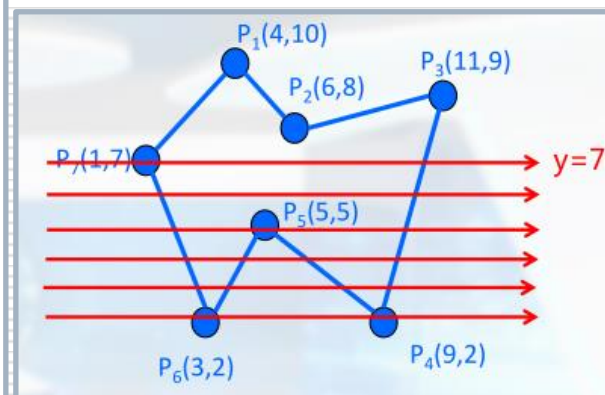
图：将多边形的某些边缩短以分离那些应计为1个交点的顶点

有效边表算法 (Y向连贯性算法)

每做一次新的扫描线时，要对已有的边进行三个处理：

- 1、是否被去除掉；
- 2、如果不被去除，第二就要对它的数据进行更新。所谓更新数据就是要更新它的x值，即： $x+1/k$
- 3、看有没有新的边进来，新的边在NET里，可以插入排序插进来。

这个算法过程从来没有求交，这套数据结构使得你不用求交点！避免了求交运算。



增量计算
删除之后的无效边
插入新边

边界标志算法

□ 基本思想：先画边界后填色

先用特殊的颜色在帧缓存中将多边形的边界勾画出来，再采用和扫描线算法类似方法 将位于多边形内的各个区段着色。对每条扫描线依从左到右的顺序逐个访问扫描线上的像素。使用一个布尔量**Inside**表示当前点是否在多边形内。（注意每一条扫描线上打标记的点的个数必须是偶数）

□ 分为两个步骤：打标记；填充。

□ 当用软件实现本算法时，速度与改进的有效边表算法相当，但本算法用硬件实现后速度会有很大提高。该算法不需要对各种表进行维护，不必排序。

边界标志算法

分为两个步骤：

(1)打标记

(2)填充

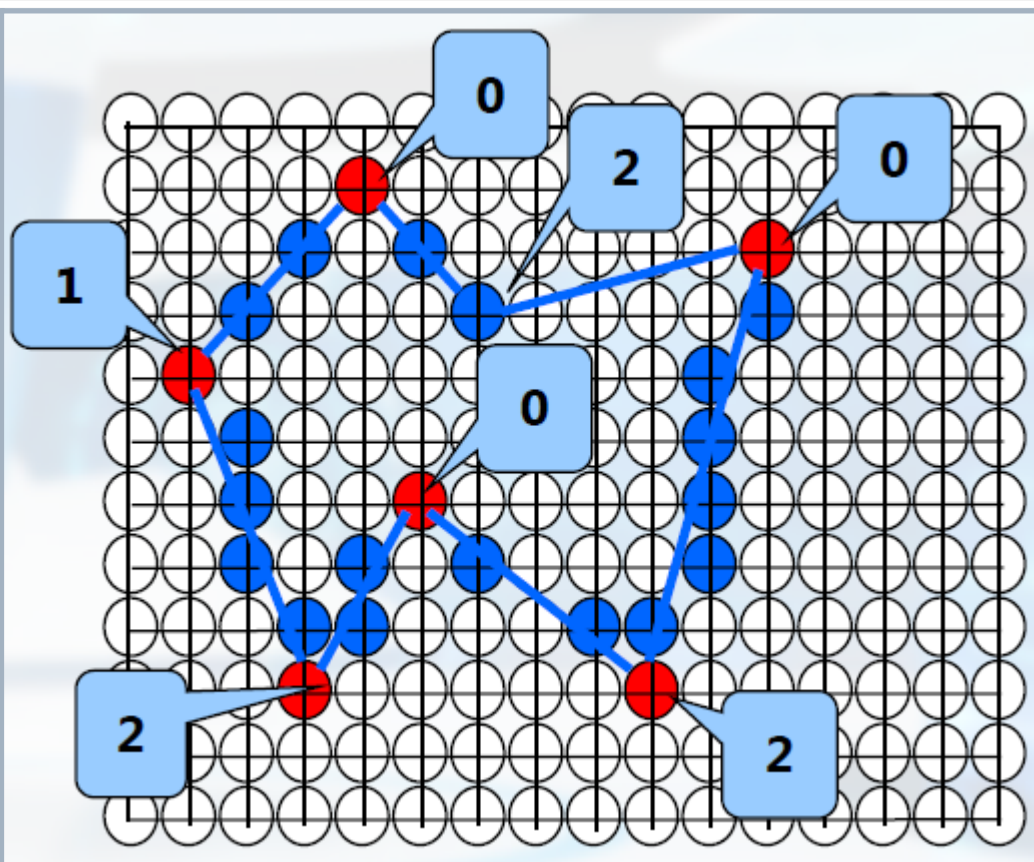
Inside

初值：假

遇到标记点：取反

真：填充；

假：不填充



在帧缓存中对多边形的每条边进行直线扫描转换，即对多边形边界所经过的像素打上标记。

边界标志算法

分为两个步骤：

(1)打标记

(2)填充

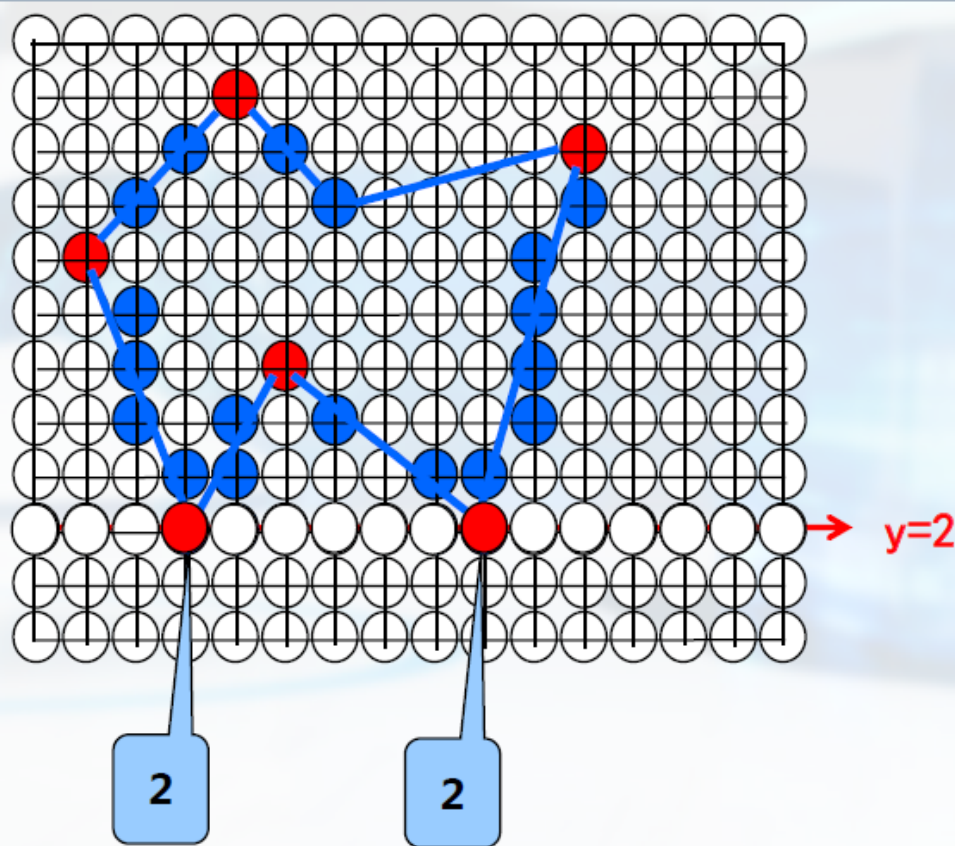
Inside

初值：假

遇到标记点：取反

真：填充；

假：不填充



对每条扫描线依从左到右的顺序逐个访问扫描线上的像素，用**Inside**表示当前点是否在多边形内，每当当当前访问的像素是被打上标记的点时，就把**inside**取反。

边界标志算法

分为两个步骤：

(1)打标记

(2)填充

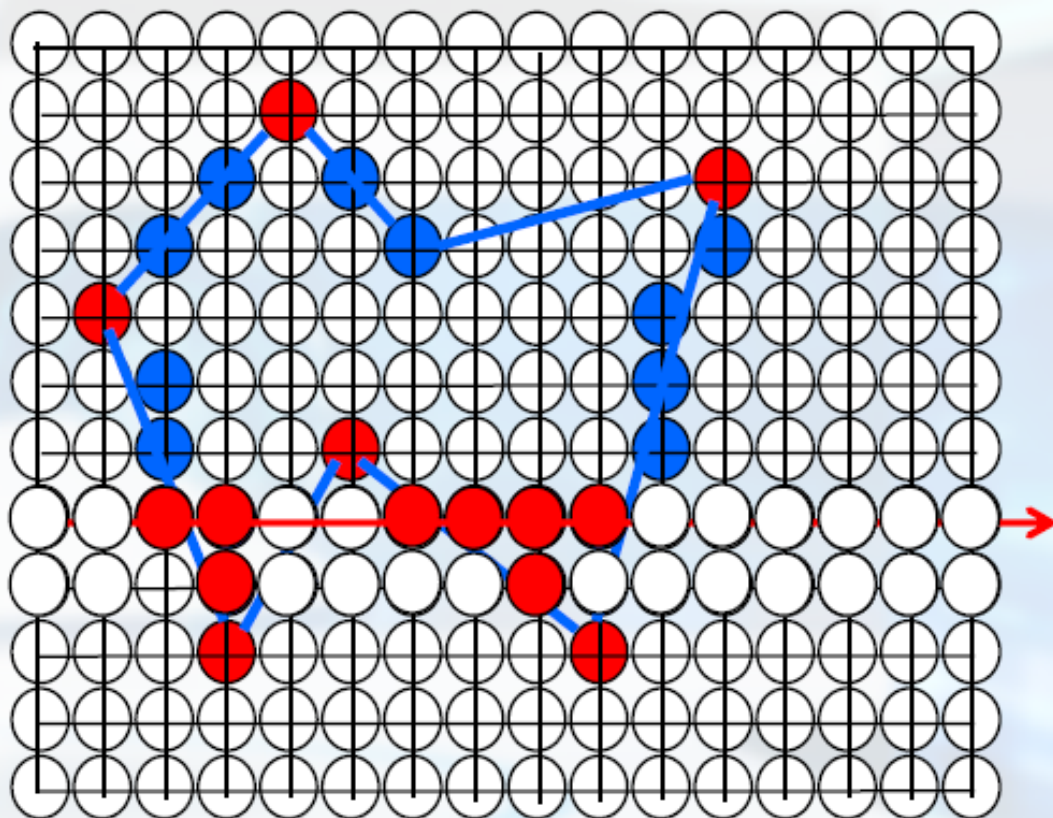
Inside

初值：假

遇到标记点：取反

真：填充；

假：不填充

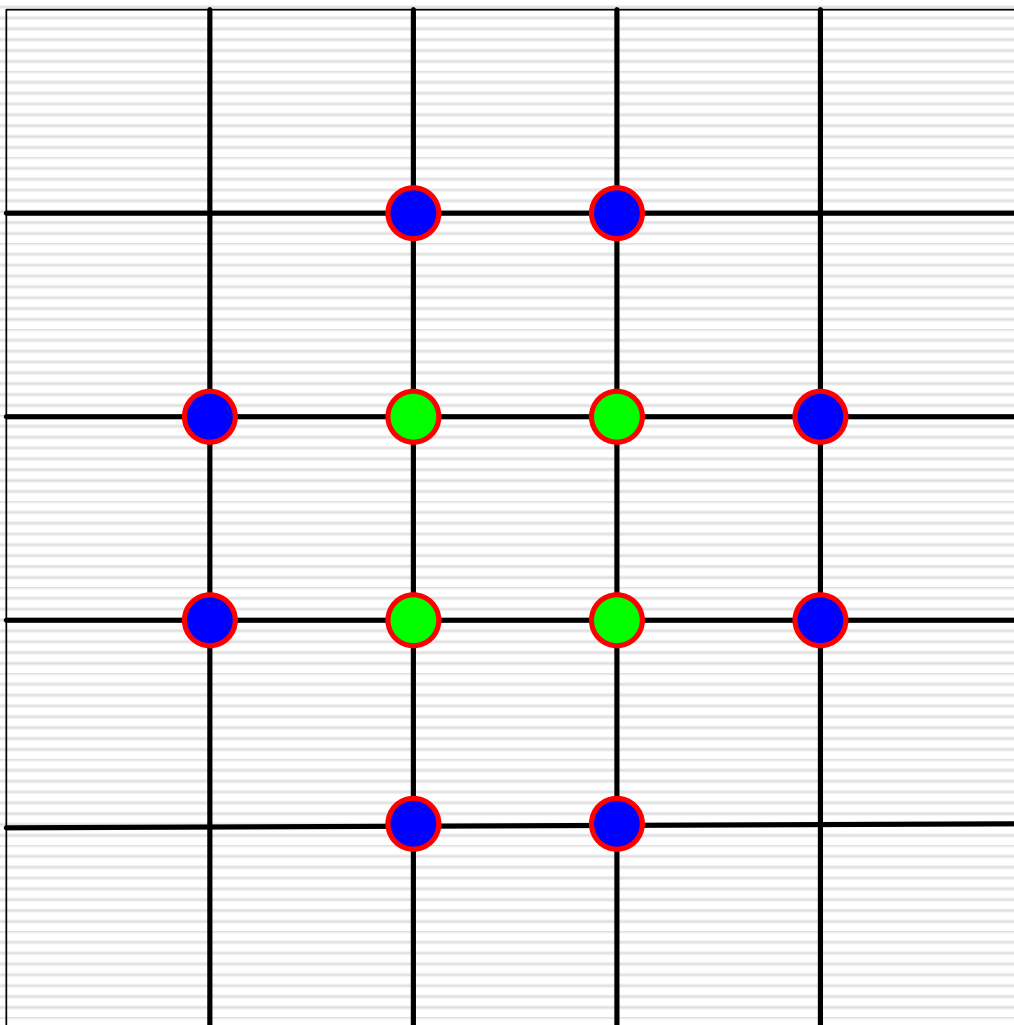


区域填充

- 基本概念
- 区域的表示方法
- 区域的分类
- 区域填充算法

基本概念

- 这里表示的**区域**是指已经表示成**点阵形式**的填充图形，它是相互连通的一组**像素的集合**。
- **区域填充**是指从区域内的某一个像素点（种子点）开始，由内向外将填充色扩展到整个区域内的过程。
 - ✓ 与多边形扫描转换基本思想不同。多边形扫描转换是指将多边形的顶点表示转化为点阵表示。
 - ✓ 区域填充只改变区域的填充颜色，不改变区域表示方法。
- 区域的表示：**边界表示法**、**内点表示法**



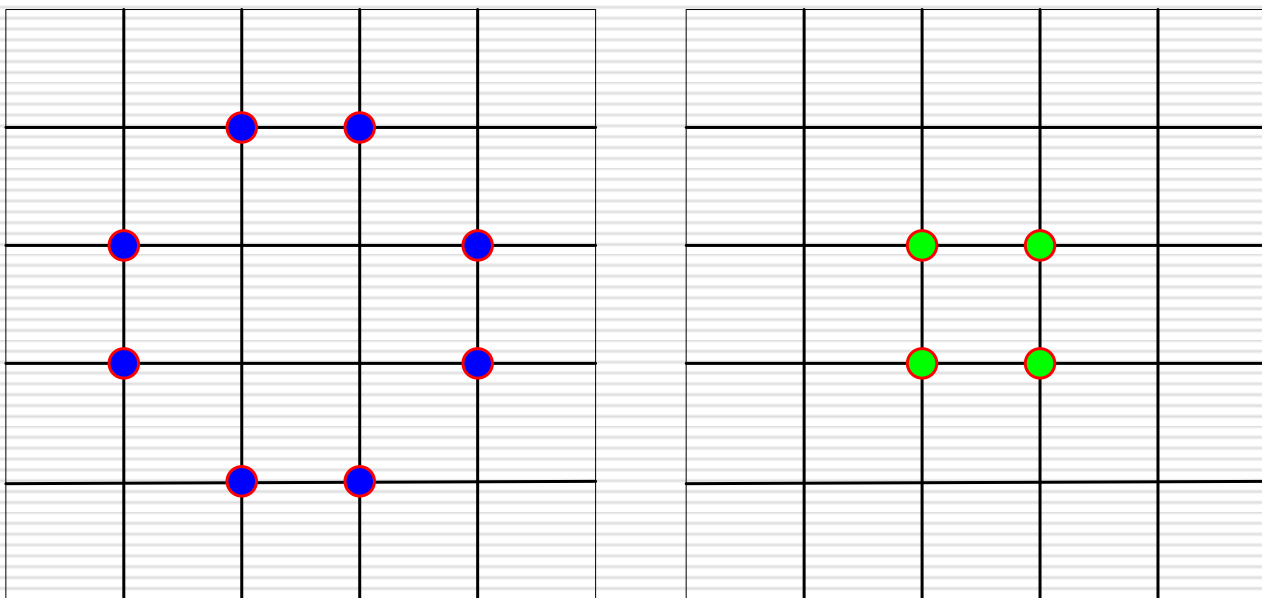
图：区域的概念

区域的表示方法

- **边界表示法**：把位于给定区域的边界上的像素一一列举出来的方法，即**枚举出边界上的所有像素**。
- 边界表示法中，由于边界由特殊颜色指定，填充算法可以逐个像素地向外处理，直到遇到边界颜色为止，这种方法称为**边界填充算法**（**Boundary-fill Algorithm**）。

□ **内点表示法**：枚举出给定区域内所有象素的表示方法。

以内点表示法为基础的区域填充算法称为**泛填充算法**
(**Flood-fill Algorithm**)。



图：区域的表示方法

区域的分类

区域填充算法的前提是区域是**连通**的，区域可以分成**4-连通区域**和**8-连通区域**：

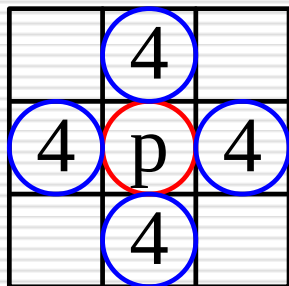
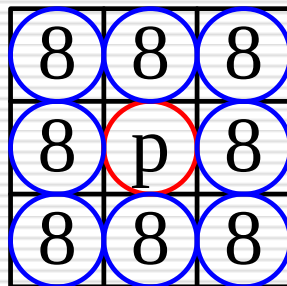


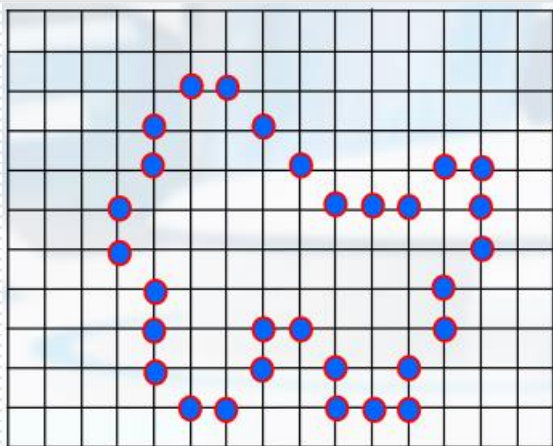
图 (a) 4-邻接点



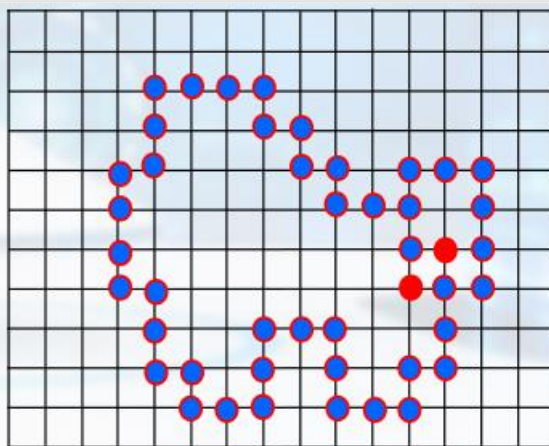
(b) 8-邻接点

- **4-连通区域**：从区域上的一点出发，通过访问已知点的**4-邻接点**，在**不越出区域**的前提下，遍历区域内的所有像素点。
- **8-连通区域**：从区域上的一点出发，通过访问已知点的**8-邻接点**，在**不越出区域**的前提下，遍历区域内的所有像素点。
- ◆ **4连通区域通常可以看作是8连通区域，但对边界有要求！**

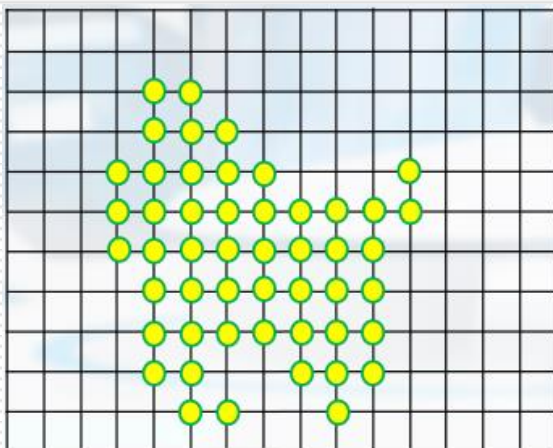
4连通与8连通区域的区别



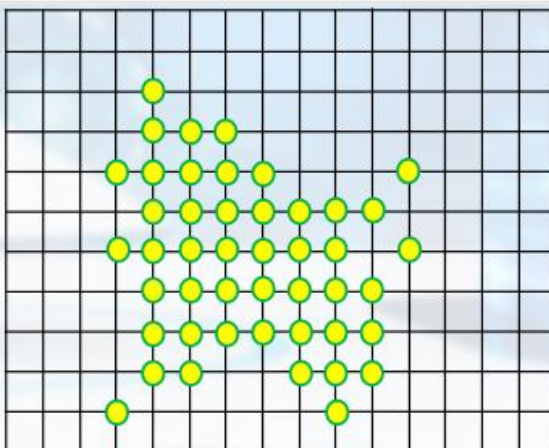
4连通边界表示



8连通边界表示



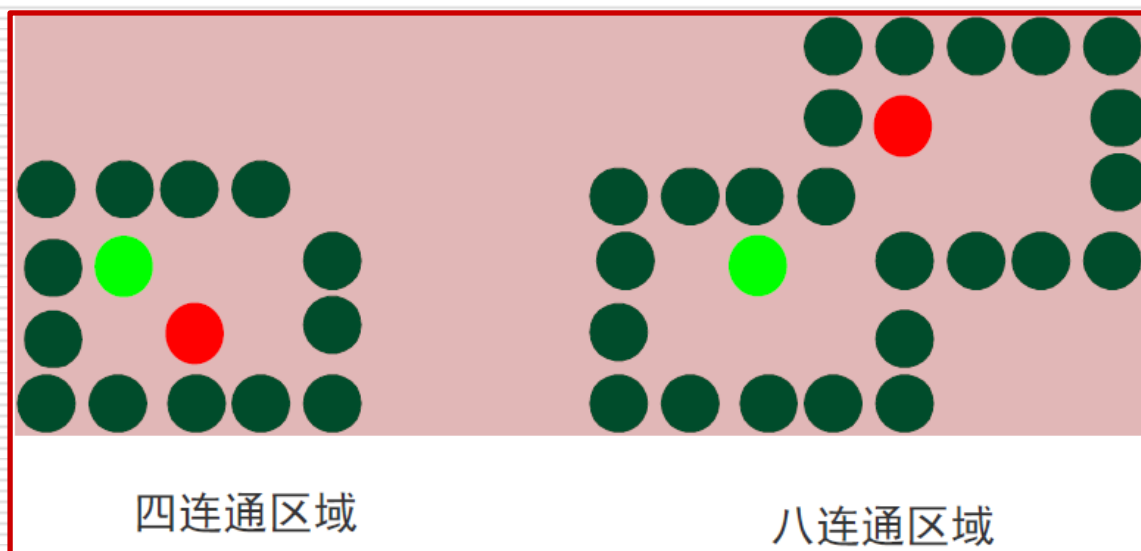
4连通内点表示



8连通内点表示

- 对于左边边界表示的区域，内部点可以通过**4**-邻接点遍历，也可通过**8**-邻接点遍历，但这个区域仅是一个**4**-连通区域，而非**8**-连通区域。因为在内部点进行**8**-邻接点遍历时，可能会越出区域。
- 边界表示的**4**-连通区域不一定是一个**8**-连通区域。
- 边界表示的**8**-连通区域不一定是一个**4**-连通区域。
- 内点表示的**4**-连通区域也是**8**-连通区域。
- 内点表示的**8**-连通区域则不一定是**4**-连通区域。

4连通与8连通区域的区别



8连通区域比4连通区域边界要求高！

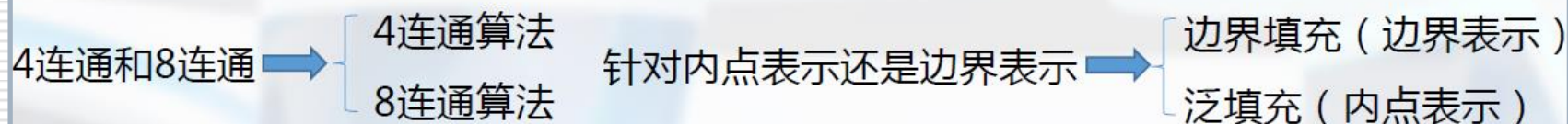
对于一个4连通边界表示的区域，需要从区域内一点出发，通过访问已知点的4-邻接点，在不越出区域的前提下，遍历区域内的所有像素。

对于8连通边界表示的区域，需要通过访问区域内已知点的8-邻接点来遍历整个区域。

区域填充算法——种子填充算法

- 区域填充算法：是根据区域内的一个已知像素点（种子点）出发，找到区域内其他像素点的过程，所以把这一类算法也成为**种子填充算法**。

区域填充算法的分类：



区域填充算法——边界填充算法

- 算法的输入：种子点坐标(x, y)，填充色以及边界颜色。
- 利用堆栈实现简单的种子填充算法
算法从种子点开始检测相邻位置是否是边界颜色，若不是就用填充色着色，并检测该像素点的相邻位置，直到检测完区域边界颜色范围内的所有像素为止。

区域填充算法——边界填充算法

栈结构实现4-连通边界填充算法的算法步骤为：

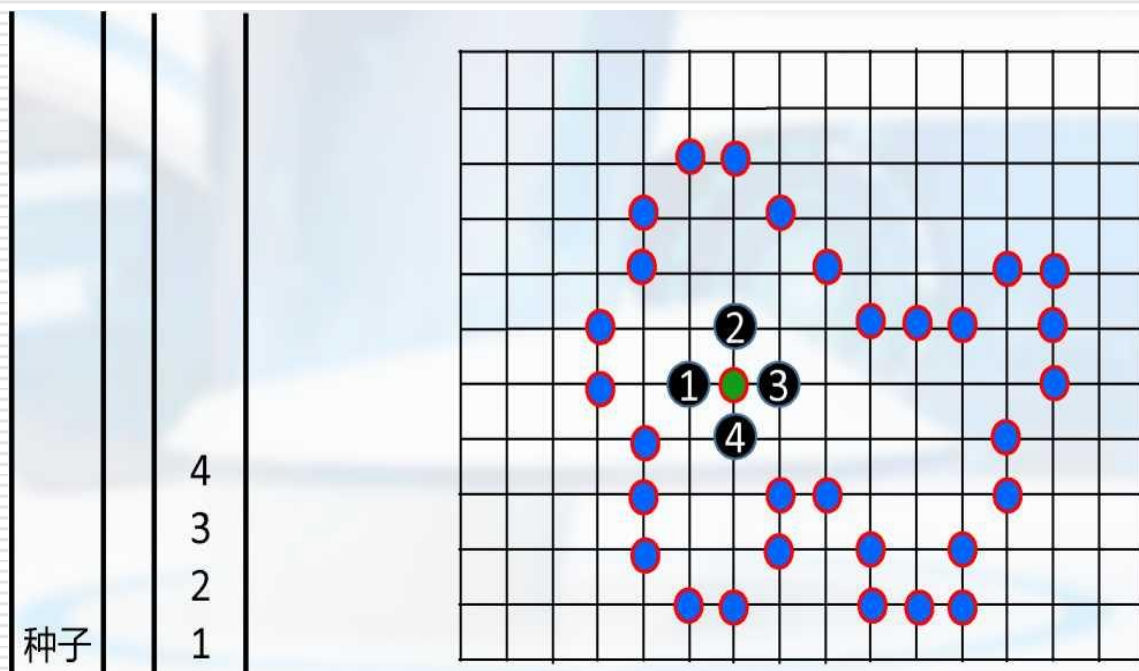
种子像素入栈；当栈非空时重复执行如下三步操作：

(a)栈顶像素出栈；

(b)将出栈像素置成填充色；

(c)检查出栈像素的4-邻接点，若其中某个像素点不是边界色且未置成填充色，则把该像素入栈。

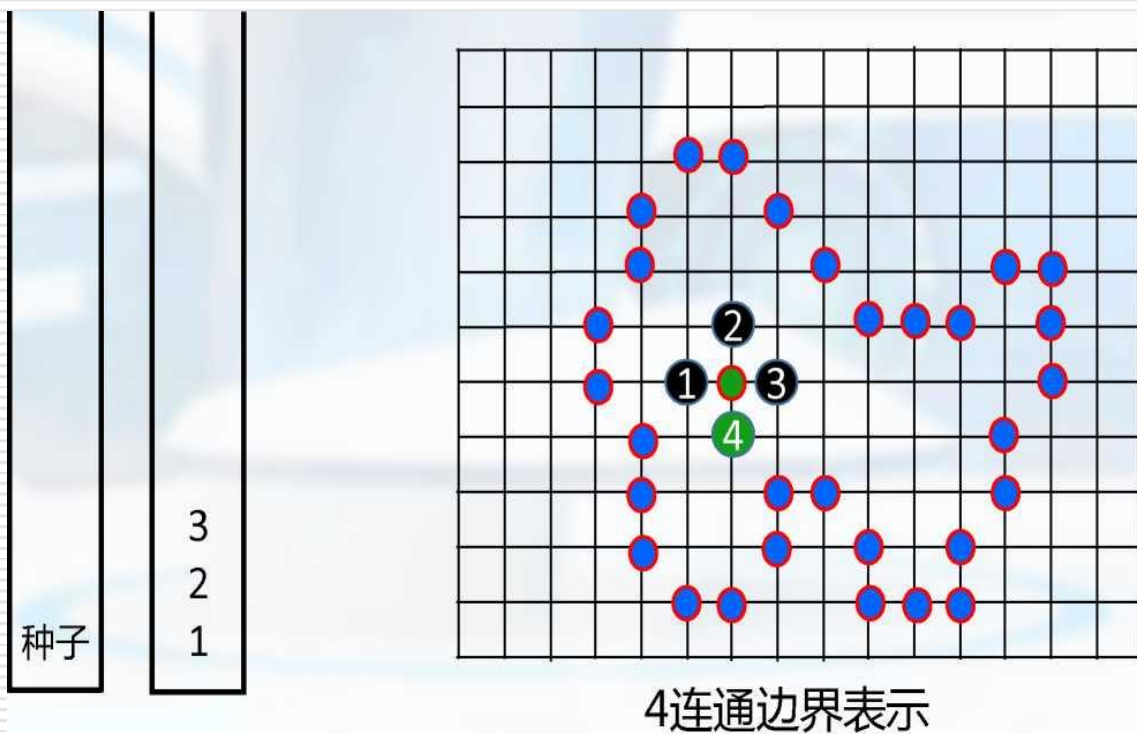
简单的种子填充算法(4-连通边界填充算法)示例



4连通边界表示

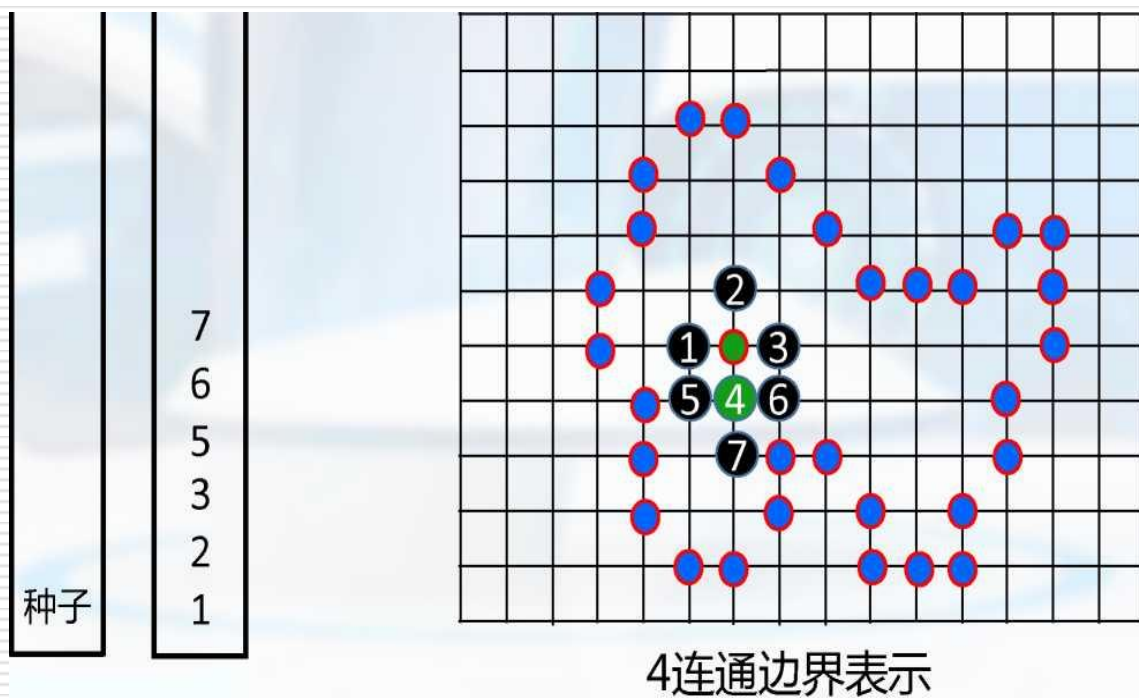
种子填充思想

以边界表示为例看种子
填充思想:



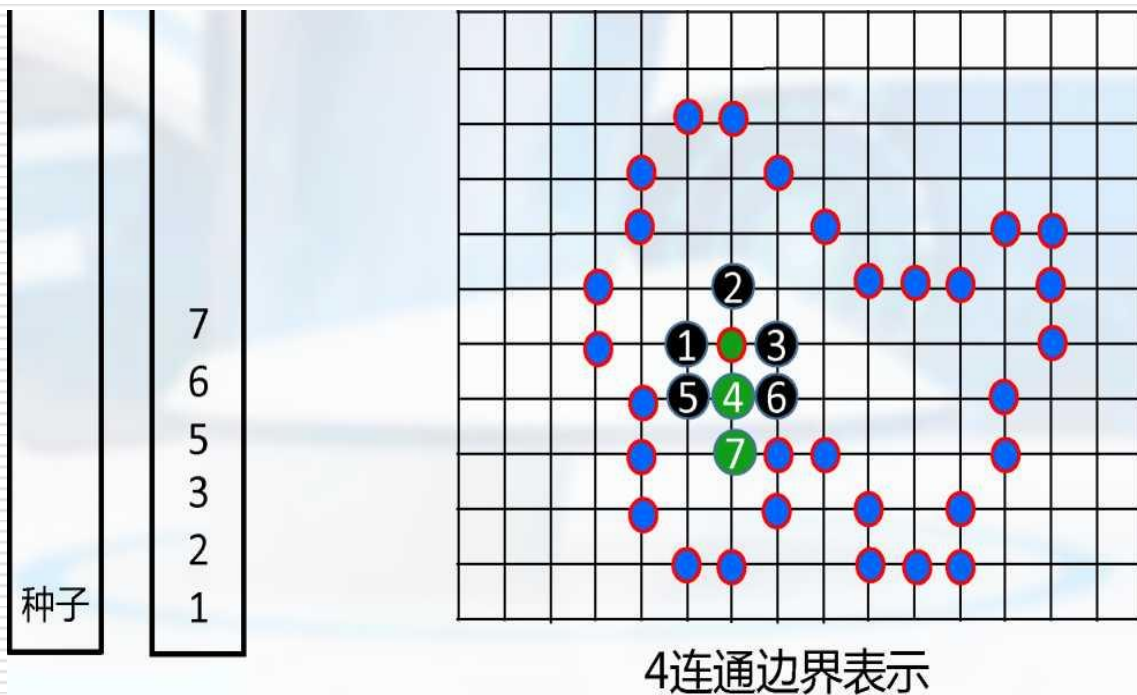
种子填充思想

以边界表示为例看种子填充思想:



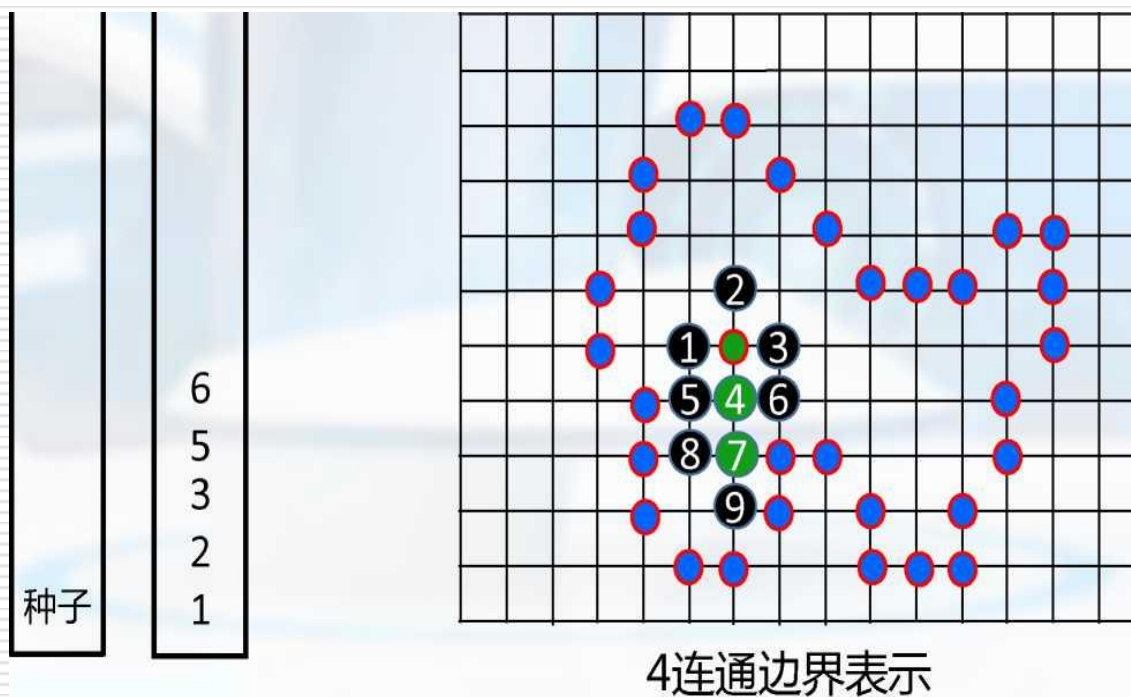
种子填充思想

以边界表示为例看种子填充思想:



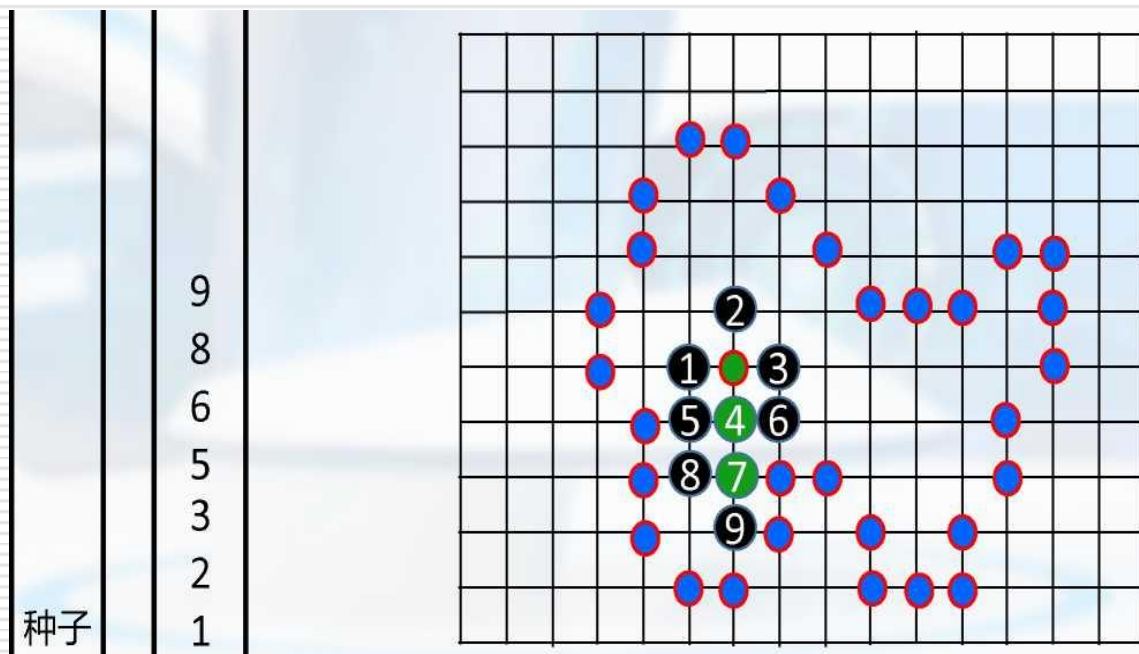
种子填充思想

以边界表示为例看种子填充思想:



种子填充思想

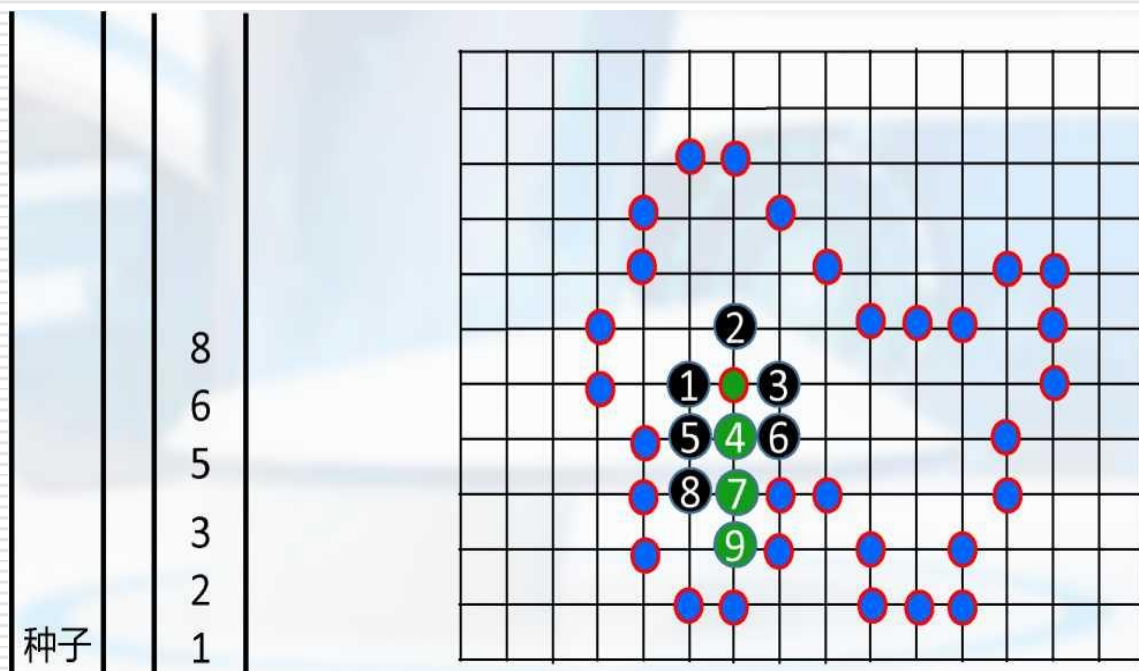
以边界表示为例看种子填充思想:



4连通边界表示

种子填充思想

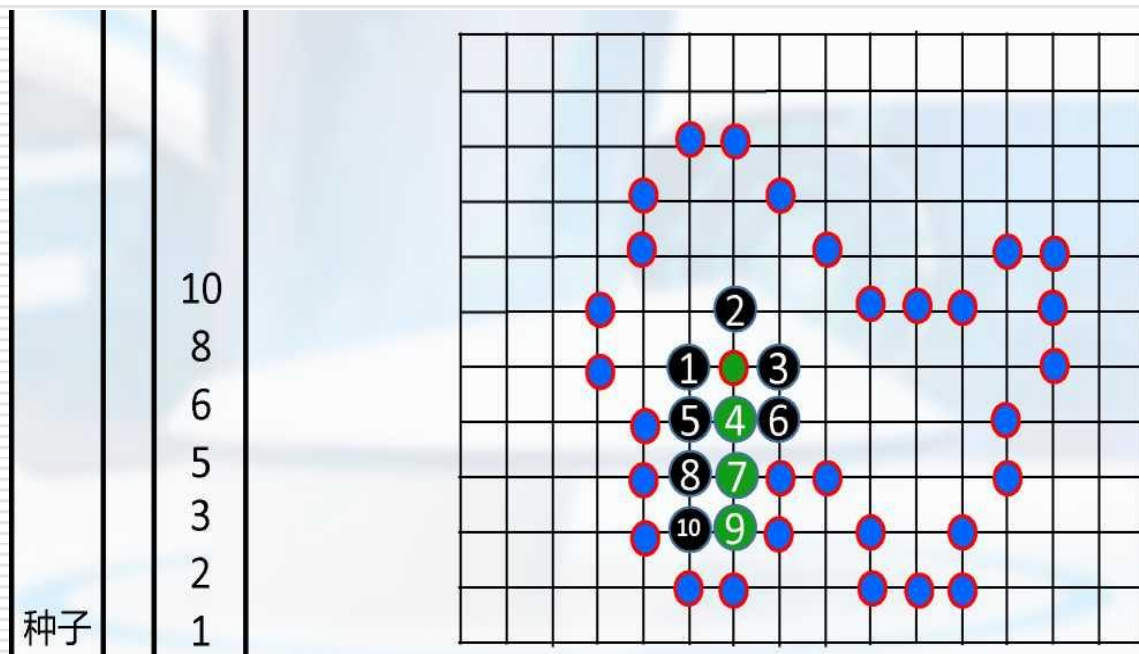
以边界表示为例看种子填充思想:



4连通边界表示

种子填充思想

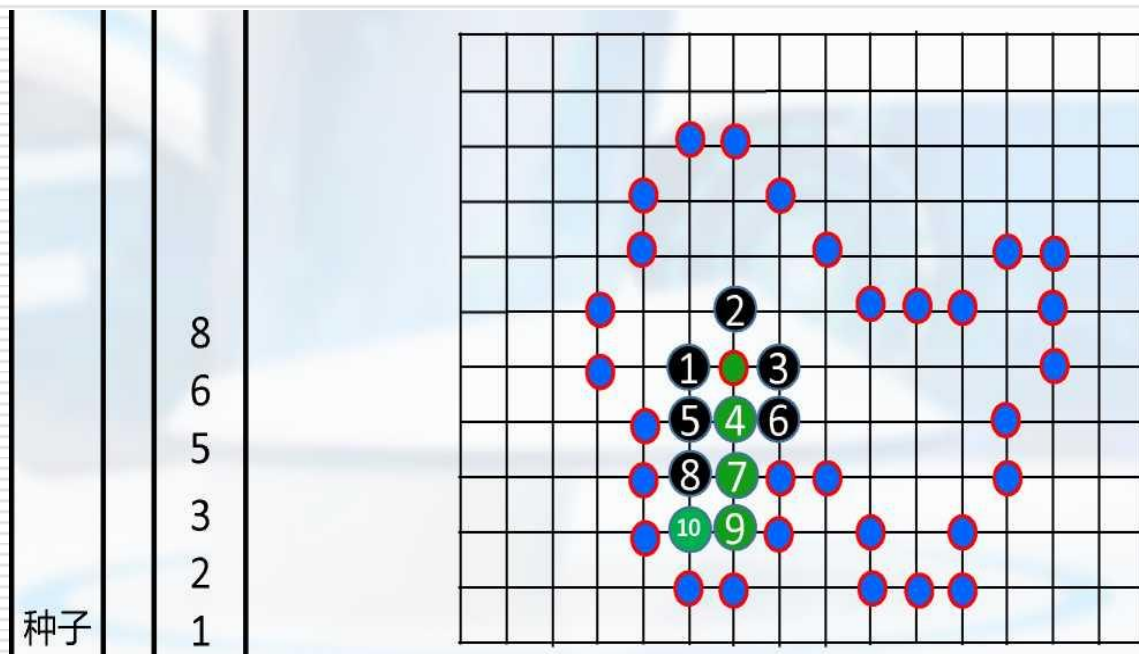
以边界表示为例看种子填充思想:



4连通边界表示

种子填充思想

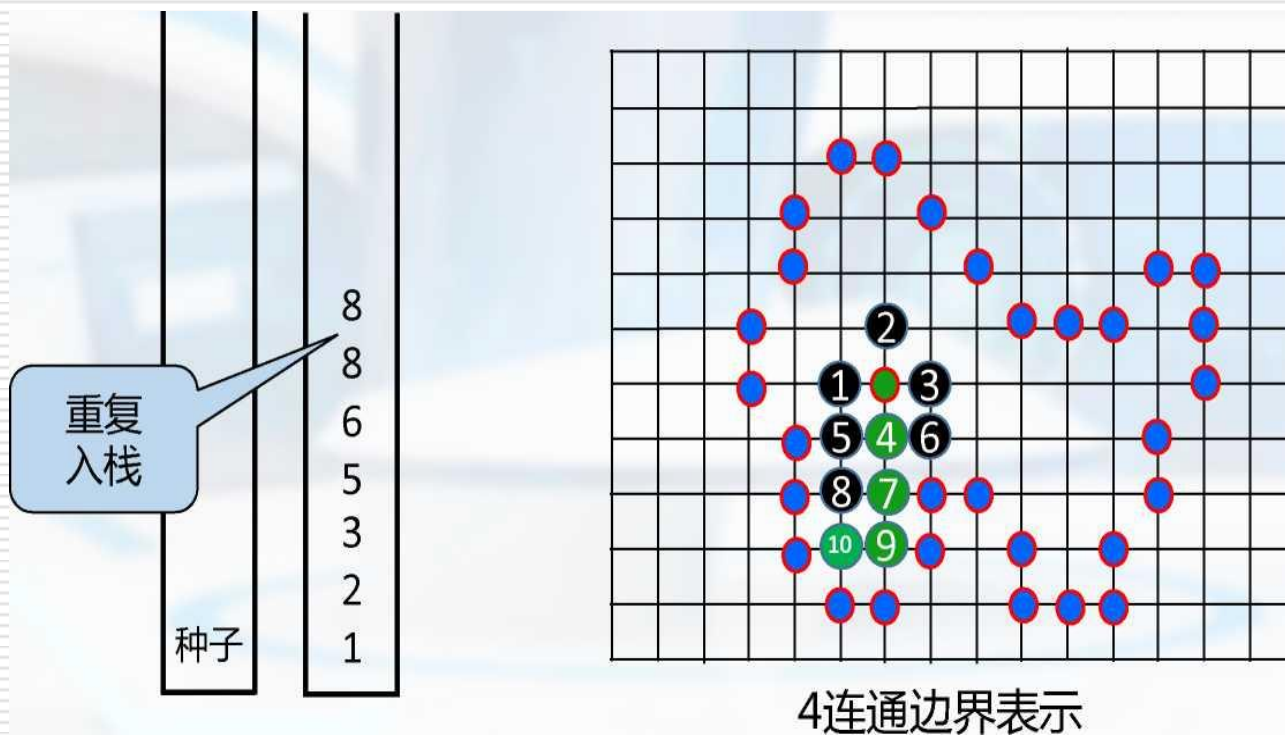
以边界表示为例看种子
填充思想:



4连通边界表示

种子填充思想

以边界表示为例看种子填充思想:



区域填充算法——边界填充算法

栈结构实现8-连通边界填充算法的算法步骤为：

种子像素入栈；当栈非空时重复执行如下三步操作：

(a) 栈顶像素出栈；

(b) 将出栈像素置成填充色；

(c) 检查出栈像素的8-邻接点，若其中某个像素点不是边界色且未置成填充色，则把该像素入栈。

区域填充算法——泛填充算法

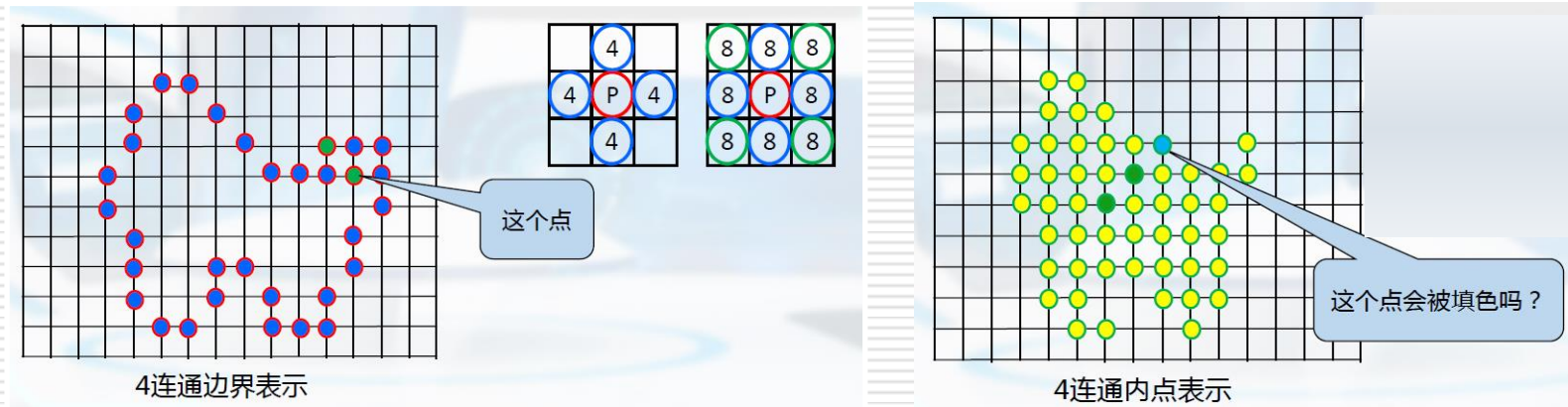
- 以内点表示法为基础的区域填充算法称为泛填充算法。
- 泛填充算法通常用于给区域重新着色。
- 算法的输入：种子点坐标 (x, y) ，填充色和内部点的颜色。
- 算法原理：算法从指定的种子 (x, y) 开始，用所希望的填充颜色赋给所有当前为给定内部颜色的像素点。

区域填充算法——泛填充算法

- 栈结构实现4-连通泛填充算法的算法步骤为：
种子像素入栈；栈非空时重复执行如下三步操作：
 - (1)栈顶像素出栈；
 - (2)将出栈像素置成填充色；
 - (3)检查出栈像素的4-邻接点，若其中某个像素点是给定内部点的颜色且未置成新的填充色，则把该像素入栈。
- 8-连通泛填充算法过程类似

区域填充算法——泛填充算法

- 当以边界表示时，4-连通边界填充算法只能填充4-连通区域，8-连通边界填充算法也只能填充8-连通区域。
 - 8-连通边界填充算法不能填充4-连通边界表示区域（如左边图）



- 当以内点表示时，8-连通泛填充算法可以填充8-连通区域也可以填充4-连通区域（如右边图），当然4-连通泛填充算法还是只能填充4-连通区域。

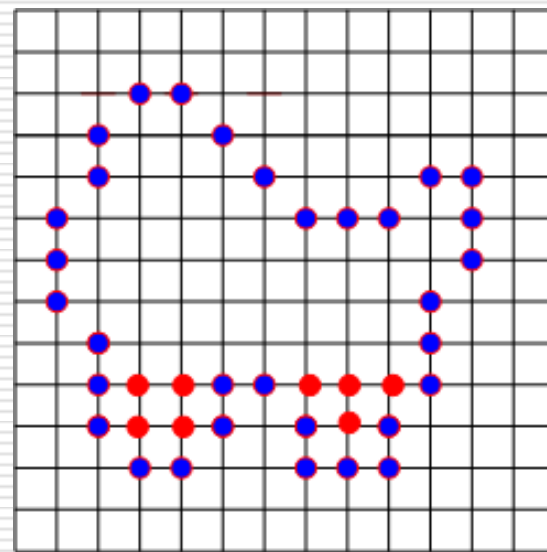
种子填充算法的不足之处

- ❑ 把太多的像素压入堆栈，降低了效率，同时需要较大的存储空间。
- ❑ 递归执行，算法简单，但效率不高，区域内每一像素都引起一次递归，进/出栈费时费内存。
- ❑ 通过沿**扫描线填充**水平像素段，来代替处理**4-邻接点**和**8-邻接点**。

扫描线种子填充算法（区域填充扫描线算法）

——边界填充算法

- 扫描线种子填充算法：
扫描线通过在任意不
间断扫描线区间中只
取一个种子像素的方
法使堆栈的尺寸极小
化。不间断区间是指
在一条扫描线上的一
组相邻像素。

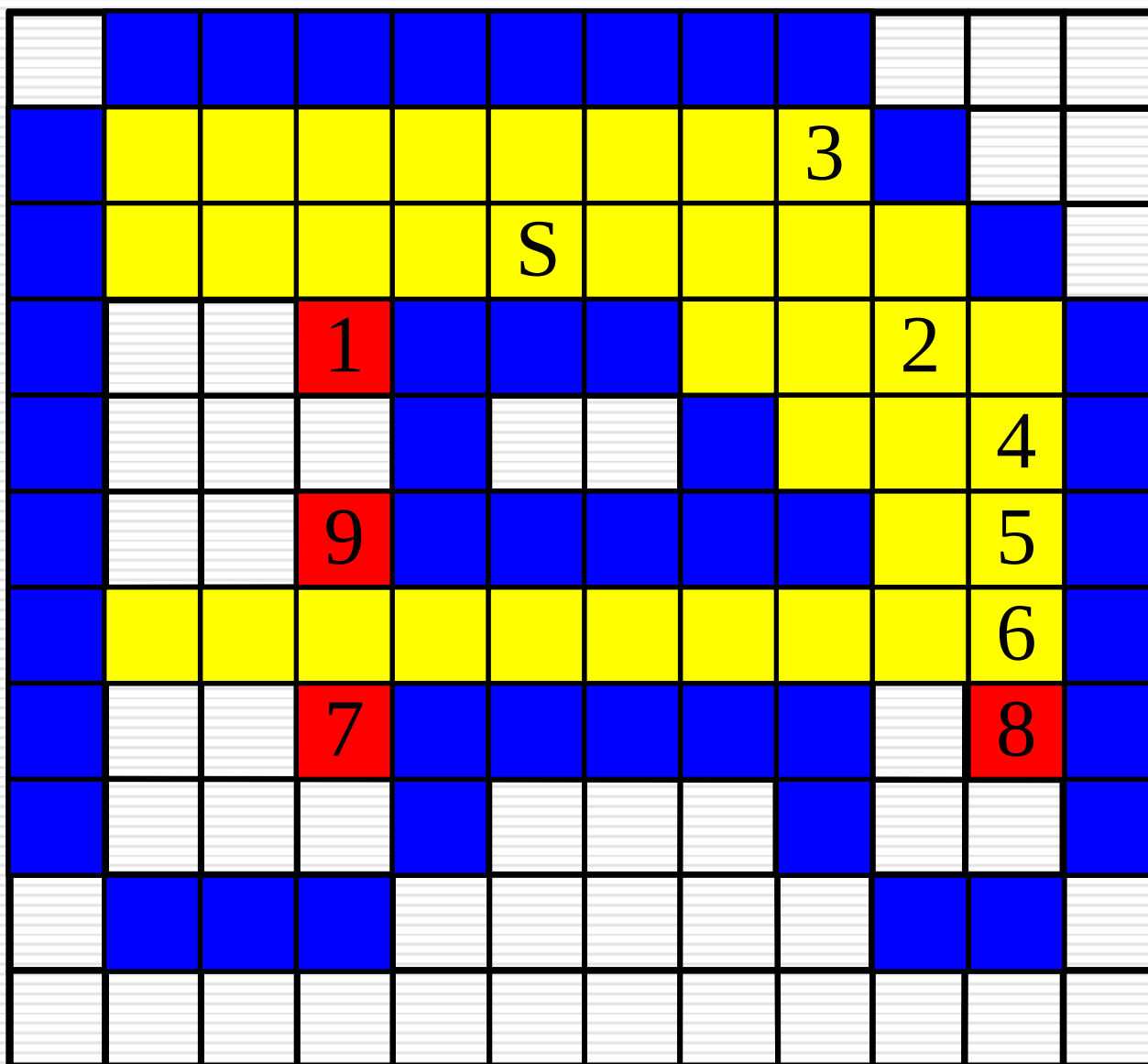


图：扫描线种子填充算法

扫描线种子填充算法——边界填充算法

- 基本过程：当给定种子点时，首先填充种子点所在的扫描线上的位于给定区域的一个区段，然后确定与这一区段相通的上下两条扫描线上位于给定区域内的区段，并依次保存下来。反复这个过程，直到填充结束。

检查上下两条扫描线上位于给定区段内的未被填充的连续水平像素段。



图：扫描线种子填充算法过程

字符处理

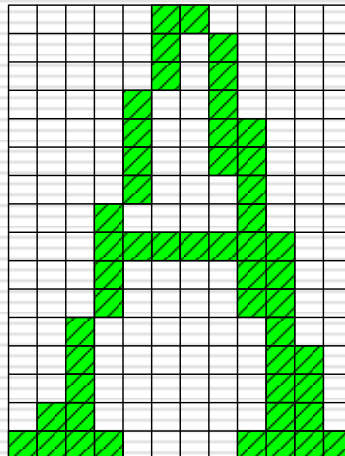
- ❑ **ASCII码**：“美国信息交换用标准代码集” (**American Standard Code for Information Interchange**), 简称**ASCII码**。
- ❑ **国标码**：“中华人民共和国国家标准信息交换编码，简称为国标码，代号**GB2312—80**。
- ❑ **字库**：字库中储存了每个字符的图形信息。
(为了在显示器等输出设备上输出字符，系统中必须装备有相应的字库)
- ❑ **点阵字库和矢量字库**。

字符处理——点阵字符

- 在点阵表示中，每个字符由一个点阵位图来表示。
- 显示时：形成字符的像素图案。

0	0	0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	1	0	1	0	0	0	0
0	0	0	0	0	1	0	1	0	0	0	0
0	0	0	0	1	0	0	1	0	0	0	0
0	0	0	0	1	0	0	1	1	0	0	0
0	0	0	0	1	0	0	1	1	0	0	0
0	0	0	0	1	0	0	0	1	0	0	0
0	0	0	1	0	0	0	0	1	0	0	0
0	0	0	1	1	1	1	1	1	1	0	0
0	0	0	1	0	0	0	0	1	1	0	0
0	0	0	1	0	0	0	0	1	1	0	0
0	0	1	0	0	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	0	0	1	1	0
0	1	1	0	0	0	0	0	0	1	1	0
1	1	1	1	0	0	0	0	1	1	1	1

(a)字符A的点阵位图



(a)字符A的像素图案

图: 字符A的点阵表示

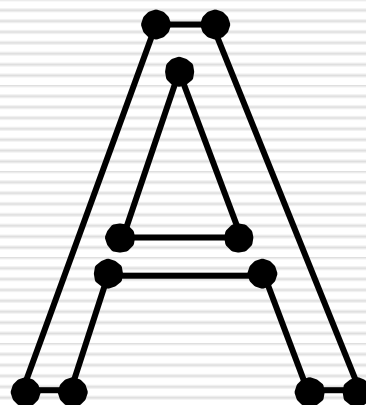
字符处理——点阵字符

- ❑ 定义和显示直接、简单。
- ❑ 存储需要耗费大量空间。
 - 采用压缩技术，如典型的轮廓字型压缩法。

轮廓字型法采用直线或二、三次曲线的集合来描述一个字符的轮廓线，轮廓线构成了一个或若干个封闭区域，显示时只要填充这些区域就可产生相应的字符。

字符处理——矢量字符

- ❑ 矢量字符采用直线和曲线段来描述字符形状，矢量字符库中记录的是**笔划信息**，而不是整个位图。
- ❑ 显示时：解释字符的每个笔划信息。占用空间少，美观，变换方便。



图：字符A的矢量表示

在OpenGL中绘图

- 点的绘制
- 直线的绘制
- 多边形面的绘制

点的绘制

□ 点的绘制

```
glBegin(GL_POINTS);  
    glVertex3f(0.0f, 0.0f, 0.0f);  
    glVertex3f(10.0f, 0.0f, 0.0f);  
glEnd();
```

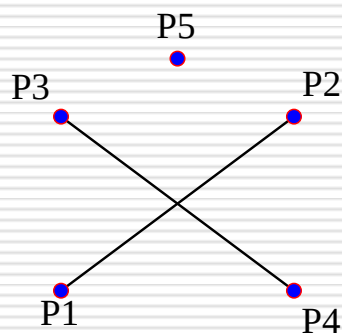
□ 点的属性（大小）

```
void glPointSize(GLfloat size);
```

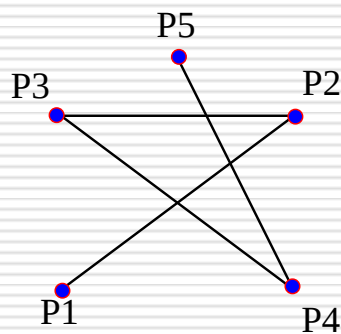

直线的绘制

□ 直线的绘制模式

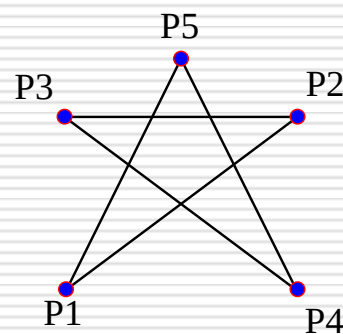
- **GL_LINES** // 绘制单条或多条线段, 每一对顶点被解释为一条直线
- **GL_LINE_STRIP** // 绘制依次连接的点组成的线, 一系列的连续直线
- **GL_LINE_LOOP** // 绘制圈, 首尾连接



(a) GL_LINES画线模式



(b) GL_LINE_LOOP画线模式



(c) GL_LINE_STRIP画线模式

图5-57 OpenGL画线模式

直线的绘制

□ 直线的属性

■ 线宽

void glLineWidth(GLfloat width)

■ 线型

glEnable(GL_LINE_STIPPLE);
glLineStipple(GLint factor, GLushort pattern);

多边形面的绘制

□ 三角形面的绘制

- **GL_TRIANGLES**

- **GL_TRIANGLE_STRIP**

- **GL_TRIANGLE_FAN**

□ 四边形面的绘制

- **GL_QUADS**

- **GL_QUADS_STRIP**

□ 多边形面的绘制 (**GL_POLYGON**)